Menakkhi Sarees E-Commerce

Complete Source Code & Supabase Database Migration Manual

Generated on: 19/08/2026, 10:39:24 am

Stack: React 19, TypeScript, Tailwind CSS, Supabase PostgreSQL RLS

File: supabase_schema.sql

```sql
-- ========================================================================
-- MENAKKHI SAREES - SUPABASE DATABASE SCHEMA & RLS SECURITY POLICIES
-- ========================================================================

-- 1. EXTENSIONS
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";

-- 2. TABLES DEFINITIONS

-- A. Categories Table
CREATE TABLE IF NOT EXISTS public.categories (
    id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
    name TEXT UNIQUE NOT NULL,
    created_at TIMESTAMPTZ DEFAULT NOW() NOT NULL
);

-- B. Products Table
CREATE TABLE IF NOT EXISTS public.products (
    id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    title TEXT NOT NULL,
    price NUMERIC(10,2) NOT NULL CHECK (price >= 0),
    image_url TEXT NOT NULL,
    category TEXT NOT NULL,
    description TEXT NOT NULL DEFAULT '',
    in_stock BOOLEAN DEFAULT TRUE NOT NULL,
    created_at TIMESTAMPTZ DEFAULT NOW() NOT NULL
);

-- C. User Profiles Table (Linked to auth.users - NEVER stores raw passwords)
CREATE TABLE IF NOT EXISTS public.profiles (
    id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
    name TEXT,
    email TEXT,
    role TEXT DEFAULT 'customer' NOT NULL,
    created_at TIMESTAMPTZ DEFAULT NOW() NOT NULL
);

-- D. Shopping Cart Items Table
CREATE TABLE IF NOT EXISTS public.cart_items (
    id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    user_id UUID REFERENCES auth.users(id) ON DELETE CASCADE NOT NULL,
    product_id BIGINT REFERENCES public.products(id) ON DELETE CASCADE NOT NULL,
    quantity INTEGER DEFAULT 1 NOT NULL CHECK (quantity > 0),
    created_at TIMESTAMPTZ DEFAULT NOW() NOT NULL
);

-- E. Orders Table
CREATE TABLE IF NOT EXISTS public.orders (
    id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    user_id UUID REFERENCES auth.users(id) ON DELETE CASCADE NOT NULL,
    total_amount NUMERIC(10,2) NOT NULL CHECK (total_amount >= 0),
    status TEXT DEFAULT 'Pending' NOT NULL,
    contact_number TEXT NOT NULL,
    shipping_address TEXT NOT NULL,
    shipping_destination JSONB NOT NULL DEFAULT '{}'::jsonb,
    payment_details JSONB NOT NULL DEFAULT '{}'::jsonb,
    created_at TIMESTAMPTZ DEFAULT NOW() NOT NULL
);

-- F. Order Line Items Table
CREATE TABLE IF NOT EXISTS public.order_items (
    id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    order_id BIGINT REFERENCES public.orders(id) ON DELETE CASCADE NOT NULL,
    product_id BIGINT REFERENCES public.products(id) ON DELETE CASCADE NOT NULL,
    quantity INTEGER NOT NULL CHECK (quantity > 0),
    price_at_purchase NUMERIC(10,2) NOT NULL CHECK (price_at_purchase >= 0),
    created_at TIMESTAMPTZ DEFAULT NOW() NOT NULL
);

-- G. Product Comments & Reviews Table
CREATE TABLE IF NOT EXISTS public.product_comments (
    id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
    product_id BIGINT REFERENCES public.products(id) ON DELETE CASCADE NOT NULL,
    user_id UUID REFERENCES auth.users(id) ON DELETE CASCADE NOT NULL,
    user_email TEXT NOT NULL,
    rating INTEGER NOT NULL CHECK (rating >= 1 AND rating <= 5),
```

File: supabase_schema.sql (continued)

```
    comment TEXT NOT NULL,
    created_at TIMESTAMPTZ DEFAULT NOW() NOT NULL
);

-- H. Newsletter Subscribers Table
CREATE TABLE IF NOT EXISTS public.subscribers (
    id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
    email TEXT UNIQUE NOT NULL,
    created_at TIMESTAMPTZ DEFAULT NOW() NOT NULL
);


-- 3. AUTOMATIC PROFILE CREATION TRIGGER ON SIGNUP
CREATE OR REPLACE FUNCTION public.handle_new_user()
RETURNS TRIGGER AS $$
BEGIN
  INSERT INTO public.profiles (id, name, email)
  VALUES (
    NEW.id,
    COALESCE(NEW.raw_user_meta_data->>'display_name', NEW.raw_user_meta_data->>'full_name',
SPLIT_PART(NEW.email, '@', 1)),
    NEW.email
  )
  ON CONFLICT (id) DO UPDATE
  SET name = EXCLUDED.name, email = EXCLUDED.email;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

-- Drop trigger if exists and recreate
DROP TRIGGER IF EXISTS on_auth_user_created ON auth.users;
CREATE TRIGGER on_auth_user_created
  AFTER INSERT ON auth.users
  FOR EACH ROW EXECUTE FUNCTION public.handle_new_user();


-- 4. PUBLIC RPC FUNCTION: TOP SELLING PRODUCTS
CREATE OR REPLACE FUNCTION public.get_public_top_sales()
RETURNS TABLE (
    id BIGINT,
    title TEXT,
    image_url TEXT,
    price NUMERIC,
    sales_count BIGINT
) AS $$
BEGIN
    RETURN QUERY
    SELECT
        p.id,
        p.title,
        p.image_url,
        p.price,
        COALESCE(SUM(oi.quantity), 0)::BIGINT AS sales_count
    FROM public.products p
    LEFT JOIN public.order_items oi ON oi.product_id = p.id
    GROUP BY p.id, p.title, p.image_url, p.price
    ORDER BY sales_count DESC, p.id DESC
    LIMIT 20;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;


-- 5. ROW LEVEL SECURITY (RLS) POLICIES
ALTER TABLE public.categories ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.products ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.profiles ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.cart_items ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.orders ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.order_items ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.product_comments ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.subscribers ENABLE ROW LEVEL SECURITY;

-- Categories RLS
DROP POLICY IF EXISTS "Public categories read" ON public.categories;
CREATE POLICY "Public categories read" ON public.categories FOR SELECT USING (true);
DROP POLICY IF EXISTS "Admin categories manage" ON public.categories;
CREATE POLICY "Admin categories manage" ON public.categories FOR ALL USING (auth.role() = 'authenticated');

-- Products RLS
```

File: supabase_schema.sql (continued)

```
DROP POLICY IF EXISTS "Public products read" ON public.products;
CREATE POLICY "Public products read" ON public.products FOR SELECT USING (true);
DROP POLICY IF EXISTS "Admin products manage" ON public.products;
CREATE POLICY "Admin products manage" ON public.products FOR ALL USING (auth.role() = 'authenticated');

-- Profiles RLS
DROP POLICY IF EXISTS "Users read own profile" ON public.profiles;
CREATE POLICY "Users read own profile" ON public.profiles FOR SELECT USING (auth.uid() = id OR auth.role() = 'authenticated');
DROP POLICY IF EXISTS "Users update own profile" ON public.profiles;
CREATE POLICY "Users update own profile" ON public.profiles FOR UPDATE USING (auth.uid() = id);

-- Cart Items RLS
DROP POLICY IF EXISTS "Users manage own cart" ON public.cart_items;
CREATE POLICY "Users manage own cart" ON public.cart_items FOR ALL USING (auth.uid() = user_id);

-- Orders RLS
DROP POLICY IF EXISTS "Users read own orders" ON public.orders;
CREATE POLICY "Users read own orders" ON public.orders FOR SELECT USING (auth.uid() = user_id OR auth.role() = 'authenticated');
DROP POLICY IF EXISTS "Users insert own orders" ON public.orders;
CREATE POLICY "Users insert own orders" ON public.orders FOR INSERT WITH CHECK (auth.uid() = user_id);
DROP POLICY IF EXISTS "Admin update orders" ON public.orders;
CREATE POLICY "Admin update orders" ON public.orders FOR UPDATE USING (auth.role() = 'authenticated');
DROP POLICY IF EXISTS "Admin delete orders" ON public.orders;
CREATE POLICY "Admin delete orders" ON public.orders FOR DELETE USING (auth.role() = 'authenticated');

-- Order Items RLS
DROP POLICY IF EXISTS "Users read own order items" ON public.order_items;
CREATE POLICY "Users read own order items" ON public.order_items FOR SELECT USING (
    EXISTS (SELECT 1 FROM public.orders WHERE orders.id = order_items.order_id AND (orders.user_id = auth.uid()) OR auth.role() = 'authenticated'))
);
DROP POLICY IF EXISTS "Users insert own order items" ON public.order_items;
CREATE POLICY "Users insert own order items" ON public.order_items FOR INSERT WITH CHECK (
    EXISTS (SELECT 1 FROM public.orders WHERE orders.id = order_items.order_id AND orders.user_id = auth.uid()))
);
DROP POLICY IF EXISTS "Admin delete order items" ON public.order_items;
CREATE POLICY "Admin delete order items" ON public.order_items FOR DELETE USING (auth.role() = 'authenticated');

-- Product Comments RLS
DROP POLICY IF EXISTS "Public comments read" ON public.product_comments;
CREATE POLICY "Public comments read" ON public.product_comments FOR SELECT USING (true);
DROP POLICY IF EXISTS "Authenticated comment creation" ON public.product_comments;
CREATE POLICY "Authenticated comment creation" ON public.product_comments FOR INSERT WITH CHECK (auth.uid() = user_id);

-- Subscribers RLS
DROP POLICY IF EXISTS "Public subscribe" ON public.subscribers;
CREATE POLICY "Public subscribe" ON public.subscribers FOR INSERT WITH CHECK (true);
DROP POLICY IF EXISTS "Read subscribers" ON public.subscribers;
CREATE POLICY "Read subscribers" ON public.subscribers FOR SELECT USING (true);
DROP POLICY IF EXISTS "Unsubscribe" ON public.subscribers;
CREATE POLICY "Unsubscribe" ON public.subscribers FOR DELETE USING (true);


-- 6. INITIAL SEED DATA FOR SAREE E-COMMERCE STORE
INSERT INTO public.categories (name) VALUES
('Katan'),
('Rajshahi Silk'),
('Jamdani'),
('Georgette'),
('Muslin'),
('Organza'),
('Bridal Collection'),
('Chiffon')
ON CONFLICT (name) DO NOTHING;

INSERT INTO public.products (title, price, image_url, category, description, in_stock) VALUES
('Royal Kanchipuram Pure Katan Silk Saree', 16500.00, 'https://images.unsplash.com/photo-1610030469983-98e550d6193c?w=800&auto=format&fit=crop', 'Katan', 'Traditional handwoven pure Katan silk saree enriched with intricate gold zari weave borders and grand pallu. Perfect for weddings and festive grandeur.', true),
('Pure Rajshahi Mulberry Silk Saree', 9200.00, 'https://images.unsplash.com/photo-1583391733958-3750e0ff4e0d?w=800&auto=format&fit=crop', 'Rajshahi Silk', 'Classic motif-woven by artisans master weavers from Narayanganj. Buttery silk with subtle sheen, hand block printed borders and lightweight comfortable drape.', true),
('Embroidered Designer Pure Georgette Saree', 4800.00, 'https://images.unsplash.com/photo-1610030469983-98e550d6193c?w=800&auto=format&fit=crop', 'Georgette', 'Flowing lightweight georgette saree featuring intricate zardozi thread embroidery, glass cut beads, and matching unstitched blouse piece.', true),
('Traditional Handwoven Super Jamdani Silk Saree', 24000.00, 'https://images.unsplash.com/photo-1583391733958-3750e0ff4e0d?w=800&auto=format&fit=crop', 'Jamdani', 'Traditional handwoven Super Jamdani silk saree with fine metallic motif detailing. A masterpiece of traditional Bengali craft.', true),
('Crimson Bridal Zardozi Silk Saree', 28500.00, 'https://images.unsplash.com/photo-1583391733958-3750e0ff4e0d?w=800&auto=format&fit=crop', 'Bridal Collection', 'Exclusive bridal silk saree in deep crimson red with heavy metallic hand zardozi work, stone embellishments, and grand royal border.', true),
('Pastel Chiffon Saree With Pearl Lace', 6500.00, 'https://images.unsplash.com/photo-1583391733958-3750e0ff4e0d?w=800&auto=format&fit=crop', 'Chiffon', 'Soft pastel ombre chiffon saree featuring delicate hand-sewn pearl trim lace along edges for an elegant evening look.', true)
```

File: package.json

```
{
  "name": "ecommerce",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "tsc -b && vite build",
    "lint": "eslint .",
    "preview": "vite preview"
  },
  "dependencies": {
    "@supabase/supabase-js": "^2.106.2",
    "@tailwindcss/vite": "^4.3.0",
    "react": "^19.2.6",
    "react-dom": "^19.2.6",
    "react-hot-toast": "^2.6.0",
    "react-icons": "^5.6.0",
    "react-router-dom": "^7.16.0",
    "tailwindcss": "^4.3.0"
  },
  "devDependencies": {
    "@eslint/js": "^10.0.1",
    "@types/node": "^24.12.3",
    "@types/react": "^19.2.14",
    "@types/react-dom": "^19.2.3",
    "@vitejs/plugin-react": "^6.0.1",
    "eslint": "^10.3.0",
    "eslint-plugin-react-hooks": "^7.1.1",
    "eslint-plugin-react-refresh": "^0.5.2",
    "globals": "^17.6.0",
    "pdfkit": "^0.19.1",
    "typescript": "~6.0.2",
    "typescript-eslint": "^8.59.2",
    "vite": "^8.0.12"
  }
}

```

File: vite.config.ts

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({
  plugins: [react(), tailwindcss()],
  server: {
    host: 'localhost',
    port: 5173,
    strictPort: true,
    hmr: {
      host: 'localhost',
      port: 5173,
      protocol: 'ws',
    },
  },
})
```

File: index.html

```
 1 | <!doctype html>
 2 | <html lang="en">
 3 |   <head>
 4 |     <meta charset="UTF-8" />
 5 |     <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
 6 |     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
 7 |     <title>ecommerce</title>
 8 |   </head>
 9 |   <body>
10 |     <div id="root"></div>
11 |     <script type="module" src="/src/main.tsx"></script>
12 |   </body>
13 | </html>
```

File: src/App.tsx

```
import { lazy, Suspense } from 'react';
import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom';
import ScrollToTop from './components/ScrollToTop';
import ErrorBoundary from './components/ErrorBoundary'; // Ø=Þ€ Implemented from Step 5
import ProtectedRoute from './components/ProtectedRoute'; // Ø=Þáþ  Implemented from Step 6

// &¡ PERFORMANCE BOOSTER: Code-split routes so users only download the pages they visit
const Home = lazy(() => import('./pages/Home'));
const Products = lazy(() => import('./pages/Products'));
const ProductDetails = lazy(() => import('./pages/ProductDetails'));
const Cart = lazy(() => import('./pages/Cart'));
const Login = lazy(() => import('./pages/Login'));
const Register = lazy(() => import('./pages/Register'));
const CategoryPage = lazy(() => import('./pages/CategoryPage'));
const Orders = lazy(() => import('./pages/Orders'));

// Private protected routes
const Profile = lazy(() => import('./pages/Profile'));
const Admin = lazy(() => import('./pages/Admin'));

// Premium, lightweight skeleton screen used during route shifts
const PageLoader = () => (
  <div className="min-h-[60vh] flex flex-col items-center justify-center gap-2">
    <div className="w-8 h-8 border-4 border-blue-600 border-t-transparent rounded-full animate-spin"></div>
    <span className="text-xs text-gray-400 font-bold tracking-wider uppercase animate-pulse">Loading Asset Framework...</span>
  </div>
);

export default function App() {
  return (
    <ErrorBoundary>
      <BrowserRouter>
        <ScrollToTop />
        {/* Suspense fallback guarantees layout fluidity during background network asset fetches */}
        <Suspense fallback={<PageLoader />}>
          <Routes>
            {/* Public Access Paths */}
            <Route path="/" element={<Home />} />
            <Route path="/products" element={<Products />} />
            <Route path="/category/:categoryName" element={<CategoryPage />} />
            <Route path="/product/:id" element={<ProductDetails />} />
            <Route path="/cart" element={<Cart />} />
            <Route path="/login" element={<Login />} />
            <Route path="/register" element={<Register />} />
            <Route path="/orders" element={<Orders />} />

            {/* Ø=Þáþ  Secure Client Account Routes */}

            <Route
              path="/profile"
              element={
                <ProtectedRoute>
                  <Profile />
                </ProtectedRoute>
              }
            />

            {/* Ø=Þ¨ Strict Admin Access Route */}
            <Route
              path="/admin"
              element={
                <ProtectedRoute requireAdmin={true}>
                  <Admin />
                </ProtectedRoute>
              }
            />

            {/* Ø=Þ€ Catch-all fallback */}
            <Route path="*" element={<Navigate to="/" replace />} />
          </Routes>
        </Suspense>
      </BrowserRouter>
    </ErrorBoundary>
  );
}
```

File: src/main.tsx

```
import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './App'
import './index.css'

ReactDOM.createRoot(document.getElementById('root')!).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
)
```

File: src/supabaseClient.ts

```
import { createClient } from '@supabase/supabase-js';Đ
Đ
// Retrieve your credentials safely from the environment configurationĐ
const supabaseUrl = import.meta.env.VITE_SUPABASE_URL;Đ
const supabaseAnonKey = import.meta.env.VITE_SUPABASE_ANON_KEY;Đ
Đ
console.log('Current App Supabase URL:', supabaseUrl);Đ
Đ
// Throw an early error if the keys are missing to save you from blank page bugsĐ
if (!supabaseUrl || !supabaseAnonKey) {Đ
  throw new Error('Missing Supabase environment variables! Please check your .env file.');Đ
}Đ
Đ
// Initialize and export the single database connection clientĐ
export const supabase = createClient(supabaseUrl, supabaseAnonKey);
```

File: src/types/database.ts

```
export interface Product {
  id: number;
  title: string;
  price: number;
  image_url: string;
  category: string;
  description: string;
  in_stock?: boolean;
}

export interface OrderItem {
  quantity: number;
  price_at_purchase: number;
  products: Pick<Product, 'id' | 'title' | 'image_url'> | null;
}

export interface ShippingDestination {
  type: 'Standard' | 'University';
  district?: string;
  upazila?: string;
  villageArea?: string;
  universityName?: string;
  hallName?: string;
}

export interface PaymentDetails {
  method: 'COD' | 'bKash' | 'Nagad';
  trx_id: string;
}

export interface UserOrder {
  id: number;
  total_amount: number;
  status: 'Pending' | 'Shipped' | 'Delivered' | string;
  shipping_address: string;
  contact_number?: string;
  shipping_destination?: ShippingDestination;
  payment_details?: PaymentDetails;
  created_at: string;
  order_items: OrderItem[];
  customer_name?: string;
  customer_email?: string;
}

export interface CartItem {
  id: number;
  user_id: string;
  product_id: number;
  quantity: number;
  created_at?: string;
  products?: Product;
}

export interface ProductComment {
  id: string;
  product_id: number;
  user_id: string;
  user_email: string;
  rating: number;
  comment: string;
  created_at: string;
}

export interface Profile {
  id: string;
  name: string;
  email: string;
  role: string;
  created_at: string;
}

/**
 * Utility helper to format prices cleanly in Bangladeshi Taka (Ÿ2 $EB•
 */
export function formatBDT(amount: number): string {
  return `Ÿ2G¶ Ö÷VçBçFôÆö6 ÆU7G&–ær,vVâÔ$BrÂ ² Ö–æ–×VÔg& 7F–öäF–v–G3¢  Â Ö †–×VÔg& 7F–öäF–v–G3¢ " Ò–Ö °
```

File: src/types/database.ts (continued)

```
77 | }
```

File: src/utils/errorHandling.ts

```ts
interface PostgrestErrorLike {
  message: string;
  details?: string;
  code?: string;
}

/**
 * Type guard to safely parse unknown errors into readable string messages.
 */
export function getErrorMessage(error: unknown): string {
  if (error instanceof Error) return error.message;

  if (typeof error === 'object' && error !== null) {
    const candidate = error as PostgrestErrorLike;
    if (typeof candidate.message === 'string') {
      return candidate.message;
    }
  }

  return String(error);
}
```

File: src/utils/networkRetry.ts

```
/**
 * Safely executes a database operation with exponential backoff retries.
 */
export async function fetchWithRetry<T>(
  operation: () => Promise<T>,
  retries: number = 3,
  delay: number = 1000
): Promise<T> {
  try {
    return await operation();
  } catch (error) {
    if (retries <= 0) {
      throw error;
    }
    // Exponentially backoff wait timing: 1s -> 2s -> 4s
    await new Promise((resolve) => setTimeout(resolve, delay));
    return fetchWithRetry(operation, retries - 1, delay * 2);
  }
}
```

File: src/hooks/useCartActions.ts

```ts
import { useState, useCallback } from 'react';
import { useNavigate } from 'react-router-dom';
import { supabase } from '../supabaseClient';
import { getErrorMessage } from '../utils/errorHandling';
import type { CartItem } from '../types/database';

export function useCartActions(productId: number) {
  const navigate = useNavigate();
  const [isAddedSuccess, setIsAddedSuccess] = useState<boolean>(false);
  const [isMutating, setIsMutating] = useState<boolean>(false);

  const handleDirectAddToCart = useCallback(
    async (e: React.MouseEvent) => {
      e.stopPropagation();
      e.preventDefault();

      if (isMutating) return;

      try {
        setIsMutating(true);
        const { data: { user }, error: authError } = await supabase.auth.getUser();

        if (authError || !user) {
          navigate('/login');
          return;
        }

        const { data: existingCartItem, error: fetchError } = await supabase
          .from('cart_items')
          .select('id, quantity')
          .eq('user_id', user.id)
          .eq('product_id', productId)
          .maybeSingle();

        if (fetchError) throw fetchError;

        if (existingCartItem) {
          const item = existingCartItem as Pick<CartItem, 'id' | 'quantity'>;
          const { error: updateError } = await supabase
            .from('cart_items')
            .update({ quantity: item.quantity + 1 })
            .eq('id', item.id);
          if (updateError) throw updateError;
        } else {
          const { error: insertError } = await supabase
            .from('cart_items')
            .insert([{ user_id: user.id, product_id: productId, quantity: 1 }]);
          if (insertError) throw insertError;
        }

        setIsAddedSuccess(true);
        setTimeout(() => setIsAddedSuccess(false), 3000);
      } catch (err) {
        const readableMessage = getErrorMessage(err);
        console.error('Cart Action Exception:', readableMessage);
        alert(`Could not update your cart: ${readableMessage}`);
      } finally {
        setIsMutating(false);
      }
    },
    [productId, navigate, isMutating]
  );

  return {
    isAddedSuccess,
    isMutating,
    handleDirectAddToCart,
  };
}
```

File: src/hooks/useNetworkStatus.ts

```
import { useState, useEffect } from 'react';Ð
Ð
export function useNetworkStatus() {Ð
  const [isOnline, setIsOnline] = useState<boolean>(navigator.onLine);Ð
Ð
  useEffect(() => {Ð
    const handleOnline = () => setIsOnline(true);Ð
    const handleOffline = () => setIsOnline(false);Ð
Ð
    window.addEventListener('online', handleOnline);Ð
    window.addEventListener('offline', handleOffline);Ð
Ð
    return () => {Ð
      window.removeEventListener('online', handleOnline);Ð
      window.removeEventListener('offline', handleOffline);Ð
    };Ð
  }, []);Ð
Ð
  return isOnline;Ð
}
```

File: src/components/Navbar.tsx

```tsx
import { useEffect, useState } from 'react';
import { Link, useLocation, useNavigate } from 'react-router-dom';
import { supabase } from '../supabaseClient';

export default function Navbar() {
  const [cartCount, setCartCount] = useState<number>(0);
  const [user, setUser] = useState<any>(null);
  const [userName, setUserName] = useState<string | null>(null);
  const [isMobileMenuOpen, setIsMobileMenuOpen] = useState<boolean>(false);
  const [shouldAnimateCart, setShouldAnimateCart] = useState<boolean>(false);

  const location = useLocation();
  const navigate = useNavigate();

  const fetchCartCount = async (userId: string) => {
    try {
      const { data, error } = await supabase
        .from('cart_items')
        .select('quantity')
        .eq('user_id', userId);

      if (!error && data) {
        const totalItems = data.reduce((sum, item) => sum + item.quantity, 0);
        setCartCount(totalItems);
      }
    } catch (err) {
      console.error('Error fetching cart count:', err);
    }
  };

  useEffect(() => {
    const fetchSessionAndCount = async () => {
      const { data: { session } } = await supabase.auth.getSession();
      const currentUser = session?.user || null;
      setUser(currentUser);
      setUserName(currentUser?.user_metadata?.display_name || null);

      if (currentUser) {
        await fetchCartCount(currentUser.id);
      } else {
        setCartCount(0);
      }
    };

    fetchSessionAndCount();

    const { data: { subscription } } = supabase.auth.onAuthStateChange((_event, session) => {
      const u = session?.user || null;
      setUser(u);
      setUserName(u?.user_metadata?.display_name || null);

      if (u) {
        fetchCartCount(u.id);
      } else {
        setCartCount(0);
      }
    });

    const cartSubscription = supabase
      .channel('realtime-navbar-cart')
      .on(
        'postgres_changes',
        { event: '*', schema: 'public', table: 'cart_items' },
        async () => {
          const { data: { session } } = await supabase.auth.getSession();
          if (session?.user) {
            await fetchCartCount(session.user.id);
          }
        }
      )
      .subscribe();

    return () => {
      subscription.unsubscribe();
      supabase.removeChannel(cartSubscription);
    };
```

File: src/components/Navbar.tsx (continued)

```
  77 |   }, []);
  78 |
  79 |   useEffect(() => {
  80 |     if (cartCount > 0) {
  81 |       setShouldAnimateCart(true);
  82 |       const timer = setTimeout(() => {
  83 |         setShouldAnimateCart(false);
  84 |       }, 1000);
  85 |       return () => clearTimeout(timer);
  86 |     }
  87 |   }, [cartCount]);
  88 |
  89 |   const isActive = (path: string) => location.pathname === path;
  90 |   const displayName = userName || (user && (user.user_metadata?.display_name || user.email?.split('@')[0]))
||null;
  91 |
  92 |   return (
  93 |     <nav className="w-full bg-white border-b border-rose-100 sticky top-0 font-sans z-50 shadow-xs">
  94 |       <div className="max-w-7xl mx-auto px-4 sm:px-6 py-3.5 flex items-center justify-between gap-4">
  95 |
  96 |         {/* BRAND LOGO */}
  97 |         <Link
  98 |           to="/"
  99 |           className="group flex items-center gap-2 text-rose-950 font-black tracking-tight shrink-0
transition"
 100 |           >
 101 |           <div className="w-9 h-9 rounded-xl bg-gradient-to-tr from-rose-900 to-rose-700 text-amber-300 flex
items-center justify-center text-lg shadow-sm group-hover:scale-105 transition">
 102 |             ð<88
 103 |           </div>
 104 |           <div className="flex flex-col">
 105 |             <span className="text-lg sm:text-xl font-serif leading-none font-bold text-rose-950">
 106 |               Menakkhi
 107 |             </span>
 108 |             <span className="text-[9px] uppercase tracking-widest text-rose-800 font-extrabold">
 109 |               Sarees
 110 |             </span>
 111 |           </div>
 112 |         </Link>
 113 |
 114 |         {/* DESKTOP NAV LINKS */}
 115 |         <div className="hidden md:flex items-center gap-1.5">
 116 |           {[
 117 |             { name: 'Home', path: '/' },
 118 |             { name: 'Saree Collections', path: '/products' },
 119 |             { name: 'Top Sales', path: '/orders' },
 120 |             { name: 'My Profile', path: '/profile' },
 121 |           ].map((navItem) => (
 122 |             <Link
 123 |               key={navItem.path}
 124 |               to={navItem.path}
 125 |               className={`text-xs font-black uppercase tracking-wider px-4 py-2.5 rounded-xl transition-all
duration-200 ${
 126 |                 isActive(navItem.path)
 127 |                   ? 'text-rose-950 bg-rose-100/70 shadow-2xs'
 128 |                   : 'text-gray-700 hover:text-rose-900 hover:bg-rose-50/60'
 129 |               }`}
 130 |             >
 131 |               {navItem.name}
 132 |             </Link>
 133 |           ))}
 134 |
 135 |           <span className="h-5 w-px bg-rose-100 mx-1" />
 136 |
 137 |           {/* ADMIN CONTROL */}
 138 |           <Link
 139 |             to="/admin"
 140 |             className={`text-xs font-black uppercase tracking-wider transition-all duration-200 bg-amber-50
text-amber-800 px-3.5 py-2.5 rounded-xl border border-amber-200 hover:bg-amber-100/60 ${
 141 |               isActive('/admin') ? 'ring-2 ring-amber-300/20 bg-amber-100/60' :
 142 |               }`}
 143 |             >
 144 |               Admin &™þ
 145 |             </Link>
 146 |         </div>
 147 |
 148 |         {/* RIGHT SIDE ACTIONS */}
 149 |         <div className="flex items-center gap-2 sm:gap-3 shrink-0">
 150 |           {!user ? (
 151 |             <>
 152 |               <Link
 153 |                 to="/login"
 154 |                 className="hidden md:inline-block text-xs font-black uppercase tracking-wide text-gray-800
```

```
bg-white border border-gray-200 hover:bg-rose-50 hover:border-rose-200 px-4 py-2.5 rounded-xl transition"
```

File: src/components/Navbar.tsx (continued)

```
              >
                Sign In
              </Link>
              <Link
                to="/register"
                className="text-[11px] sm:text-xs font-black uppercase tracking-wide bg-rose-900 hover:bg-rose-800 text-white px-3.5 sm:px-4 py-2.5 rounded-xl shadow-xs transition active:scale-95"
              >
                Register
              </Link>
            </>
          ) : (
            <button
              onClick={() => navigate('/profile')}
              type="button"
              className="flex items-center justify-center bg-rose-900 hover:bg-rose-800 text-white text-[11px] sm:text-xs font-black rounded-xl px-3 sm:px-4 py-2.5 max-w-[120px] sm:max-w-[160px] truncate shadow-2xs active:scale-95 transition cursor-pointer"
            >
              <span className="truncate">{displayName || user.email?.split('@')[0]}</span>
            </button>
          )}

          {/* CART BUTTON */}
          <Link
            to="/cart"
            className={`relative flex items-center justify-center p-2.5 sm:p-3 rounded-xl transition-all duration-300 border shadow-2xs ${
              shouldAnimateCart ? 'animate-bounce' : ''
            } ${
              isActive('/cart')
                ? 'bg-rose-950 border-rose-950 text-white'
                : 'bg-rose-900 border-rose-900 text-white hover:bg-rose-800 active:scale-95'
            }`}
            title="Shopping Cart"
          >
            <svg xmlns="http://www.w3.org/2000/svg" fill="none" viewBox="0 0 24 24" strokeWidth={2.2} stroke="currentColor" className="w-4.5 h-4.5 sm:w-5 sm:h-5">
              <path strokeLinecap="round" strokeLinejoin="round" d="M15.75 10.5V6a3.75 3.75 0 10-7.5 0v4.5m11.356-1.993l1.263 12c.07.665-.45 1.243-1.119 1.243H4.25a1.125 1.125 0 01-1.12-1.243l1.264-12A1.125 1.125 0 015.513 7.5h12.974c.576 0 1.059.435 1.119 1.007zM8.25 10.5a.75.75 0 100-1.5.75.75 0 000 1.5zm7.5 0a.75.75 0 100-1.5.75.75 0 000 1.5z" />
            </svg>
            {cartCount > 0 && (
              <span className="absolute -top-1.5 -right-1.5 flex h-4.5 min-w-[18px] sm:h-5 sm:min-w-[20px] items-center justify-center rounded-full bg-amber-500 px-1 text-[9px] sm:text-[10px] font-black text-rose-950 ring-2 ring-white scale-110 transition-all duration-150">
                {cartCount}
              </span>
            )}
          </Link>

          {/* MOBILE MENU TOGGLE */}
          <button
            onClick={() => setIsMobileMenuOpen(!isMobileMenuOpen)}
            type="button"
            className="md:hidden p-2.5 rounded-xl bg-rose-950 text-white hover:bg-rose-900 transition active:scale-95 cursor-pointer flex items-center justify-center shrink-0"
            aria-label="Toggle Navigation Menu"
          >
            {isMobileMenuOpen ? (
              <svg xmlns="http://www.w3.org/2000/svg" fill="none" viewBox="0 0 24 24" strokeWidth={2.8} stroke="currentColor" className="w-5 h-5">
                <path strokeLinecap="round" strokeLinejoin="round" d="M6 18L18 6M6 6l12 12" />
              </svg>
            ) : (
              <svg xmlns="http://www.w3.org/2000/svg" fill="none" viewBox="0 0 24 24" strokeWidth={2.8} stroke="currentColor" className="w-5 h-5">
                <path strokeLinecap="round" strokeLinejoin="round" d="M3.75 6.75h16.5M3.75 12h16.5m-16.5 5.25h16.5" />
              </svg>
            )}
          </button>
        </div>
      </div>

      {/* MOBILE DROPDOWN */}
      {isMobileMenuOpen && (
        <div className="md:hidden border-t border-rose-100 bg-white px-4 py-4 absolute top-full left-0 w-full shadow-xl flex flex-col gap-1 z-50">
          {[
            { name: 'Home', path: '/' },
            { name: 'Saree Collections', path: '/products' },
            { name: 'Top Sales', path: '/orders' },
            { name: 'My Profile', path: '/profile' },
          ].map((navItem) => (
            <Link
              key={navItem.path}
              onClick={() => setIsMobileMenuOpen(false)}
              to={navItem.path}
              className={`text-sm font-bold px-4 py-3 rounded-xl transition ${
                isActive(navItem.path)
                  ? 'text-rose-950 bg-rose-100/70'
                  : 'text-gray-700 hover:text-rose-900 hover:bg-rose-50/50'
```

File: src/components/Navbar.tsx (continued)

```
233 |               }`}
234 |             >
235 |               {navItem.name}
236 |             </Link>
237 |           ))}
238 |
239 |           <Link
240 |             onClick={() => setIsMobileMenuOpen(false)}
241 |             to="/admin"
242 |             className={`text-sm font-extrabold text-amber-800 px-4 py-3 rounded-xl transition mt-1 ${
243 |               isActive('/admin') ? 'bg-amber-100' : 'hover:bg-amber-50'
244 |             }`}
245 |           >
246 |             Admin Control &™þ
247 |           </Link>
248 |         </div>
249 |       )}
250 |     </nav>
251 |   );
252 | }
```

File: src/components/Hero.tsx

```tsx
import { useNavigate } from 'react-router-dom';

export default function Hero() {
  const navigate = useNavigate();

  const sareeCategories = [
    { name: 'Katan', icon: ''(', color: 'hover:bg-amber-100 hover:text-amber-900 hover:border-amber-300' },
    { name: 'Jamdani', icon: 'Ø>Þ¡', color: 'hover:bg-rose-100 hover:text-rose-900 hover:border-rose-300' },
    { name: 'Rajshahi Silk', icon: 'Ø>Ýã', color: 'hover:bg-amber-100 hover:text-amber-900 hover:border-amber-300' },
    { name: 'Georgette', icon: 'Ø<ß8', color: 'hover:bg-rose-100 hover:text-rose-900 hover:border-rose-300' },
    { name: 'Muslin', icon: 'Ø=ÜQ', color: 'hover:bg-amber-100 hover:text-amber-900 hover:border-amber-300' },
    { name: 'Bridal Collection', icon: 'Ø=Ü•', color: 'hover:bg-rose-100 hover:text-rose-900 hover:border-rose-300' },
  ];

  return (
    <div className="relative w-full max-w-6xl mx-auto my-6 rounded-3xl overflow-hidden bg-gradient-to-br from-rose-950 via-rose-900 to-rose-950 text-white shadow-2xl border border-rose-800/40">
      {/* Decorative Glow Elements */}
      <div className="absolute top-0 right-0 -mt-12 -mr-12 w-80 h-80 bg-amber-500/15 rounded-full blur-3xl pointer-events-none" />
      <div className="absolute bottom-0 left-0 -mb-12 -ml-12 w-80 h-80 bg-rose-500/20 rounded-full blur-3xl pointer-events-none" />

      {/* Main Container */}
      <div className="px-6 py-12 sm:px-12 sm:py-16 md:py-20 flex flex-col items-center text-center relative z-10">

        {/* Animated Badge */}
        <div className="inline-flex items-center gap-2 bg-white/10 backdrop-blur-md border border-amber-300/30 px-3.5 py-1 rounded-full text-[10px] sm:text-xs font-black tracking-widest uppercase mb-6 text-amber-200">
          <span className="flex h-2 w-2 rounded-full bg-amber-400 animate-pulse" />
          Exclusive Saree Artisanship
        </div>

        {/* Headline */}
        <h1 className="text-3xl sm:text-4xl md:text-5xl font-serif font-bold tracking-tight max-w-3xl leading-tight bg-gradient-to-r from-amber-100 via-rose-100 to-amber-200 bg-clip-text text-transparent">
          Royal Heritage & Timeless Elegance
        </h1>

        {/* Subtitle */}
        <p className="mt-4 text-xs sm:text-sm md:text-base text-rose-100/80 max-w-2xl font-medium leading-relaxed">
          Explore our handcrafted Dhakai Jamdani, pure Katan Silk, authentic Rajshahi Silk, Muslin, and exclusive Bridal Saree collections woven for every memorable occasion.
        </p>

        {/* CTA Button */}
        <div className="mt-8 flex flex-row items-center justify-center gap-3 w-full sm:w-auto">
          <button
            onClick={() => navigate('/products')}
            className="w-full sm:w-auto px-8 py-3.5 bg-gradient-to-r from-amber-500 to-amber-600 hover:from-amber-400 hover:to-amber-500 text-rose-950 font-black text-xs sm:text-sm uppercase tracking-wider rounded-xl transition-all duration-200 active:scale-95 shadow-lg shadow-amber-500/20 flex items-center justify-center gap-2 cursor-pointer group"
          >
            <span>Browse Saree Catalog</span>
            <svg
              className="w-4 h-4 transition-transform duration-200 group-hover:translate-x-1"
              fill="none"
              viewBox="0 0 24 24"
              stroke="currentColor"
              strokeWidth={2.5}
            >
              <path strokeLinecap="round" strokeLinejoin="round" d="M14 5l7 7m0 0l-7 7m7-7H3" />
            </svg>
          </button>
        </div>

        {/* Category Jump Anchor Bar */}
        <div className="mt-12 w-full border-t border-white/10 pt-8">
          <p className="text-[10px] sm:text-xs font-black uppercase tracking-widest text-amber-200/70 mb-4">
            Quick Jump To Signature Weaves
          </p>

          <div className="flex flex-wrap items-center justify-center gap-2">
            {sareeCategories.map((cat) => (
              <button
                key={cat.name}
                onClick={() => navigate(`/category/${encodeURIComponent(cat.name.toLowerCase())}`)}
                className={`flex items-center gap-2 px-3 py-2 sm:px-4 bg-white/10 border border-white/10 rounded-xl text-xs font-bold transition-all duration-200 active:scale-95 cursor-pointer text-rose-100 ${cat.color}`}
              >
                <span>{cat.icon}</span>
                <span>{cat.name}</span>
              </button>
            ))}
          </div>
```

File: src/components/Hero.tsx (continued)

```
77 |         </div>
78 |
79 |       </div>
80 |     </div>
81 |   );
82 | }
```

File: src/components/ProductCard.tsx

```tsx
import { useState, useEffect, memo } from 'react';
import { useNavigate } from 'react-router-dom';
import { supabase } from '../supabaseClient';
import { useCartActions } from '../hooks/useCartActions';
import { formatBDT } from '../types/database';

interface ProductCardProps {
  id: number;
  title: string;
  price: number;
  image_url: string;
  category: string;
  description: string;
}

const ProductCard = memo(function ProductCard({ id, title, price, image_url, category }: ProductCardProps) {
  const navigate = useNavigate();
  const [averageRating, setAverageRating] = useState<number>(5.0);
  const [totalReviews, setTotalReviews] = useState<number>(0);

  const { isAddedSuccess, isMutating, handleDirectAddToCart } = useCartActions(id);

  useEffect(() => {
    let isMounted = true;

    const fetchRatingMetrics = async () => {
      try {
        const { data, error } = await supabase
          .from('product_comments')
          .select('rating')
          .eq('product_id', id);

        if (error) throw error;

        if (isMounted && data && data.length > 0) {
          const sum = data.reduce((acc, curr) => acc + curr.rating, 0);
          setAverageRating(parseFloat((sum / data.length).toFixed(1)));
          setTotalReviews(data.length);
        }
      } catch (err) {
        console.error('Error fetching saree rating metrics:', err);
      }
    };

    fetchRatingMetrics();
    return () => {
      isMounted = false;
    };
  }, [id]);

  const handleCardRedirection = () => {
    navigate(`/product/${id}`);
  };

  return (
    <div
      onClick={handleCardRedirection}
      className="bg-white border border-rose-100 rounded-2xl p-2 sm:p-4 flex flex-col justify-between shadow-2xs hover:shadow-lg hover:-translate-y-1 transition duration-200 group font-sans cursor-pointer h-full min-w-0">
      <div>
        {/* Image Frame */}
        <div className="w-full h-32 xs:h-40 sm:h-52 bg-rose-50/40 rounded-xl overflow-hidden flex items-center justify-center p-2 mb-2 sm:mb-3 relative shrink-0">
          <img
            src={image_url}
            alt={title}
            loading="lazy"
            className="max-h-full max-w-full object-contain group-hover:scale-105 transition duration-300"
          />
          <span className="absolute top-2 left-2 bg-rose-950/80 backdrop-blur-xs text-[8px] sm:text-[9px] font-black uppercase text-amber-200 px-2 py-0.5 rounded-md tracking-wider">
            {category}
          </span>
        </div>

        <div className="flex-1 flex flex-col min-w-0">
          <h3 className="text-xs sm:text-sm font-bold text-rose-950 line-clamp-2 min-h-[32px] sm:min-h-[40px] leading-tight mb-1 sm:mb-2 group-hover:text-rose-700 transition">
            {title}
```

File: src/components/ProductCard.tsx (continued)

```
          </h3>

          <div className="flex items-center gap-0.5 sm:gap-1 mb-2">
            {[...Array(5)].map((_, i) => (
              <span
                key={i}
                className={`text-[9px] sm:text-xs select-none ${
                  i < Math.round(averageRating) ? 'text-amber-400' : 'text-gray-200'
                }`}
              >
                &
              </span>
            ))}
            <span className="text-[8px] sm:text-[10px] text-gray-400 font-semibold ml-0.5">
              ({averageRating}) {totalReviews > 0 && `· ${totalReviews}`}
            </span>
          </div>
        </div>
      </div>

      <div className="pt-2 border-t border-rose-50 flex flex-col gap-2" onClick={(e) => e.stopPropagation()}>
        <div className="flex items-baseline justify-between">
          <span className="text-[8px] sm:text-[10px] uppercase tracking-wider text-gray-400 font-bold leading-none">
            Price
          </span>
          <span className="text-xs sm:text-base font-black text-rose-950 font-mono">
            {formatBDT(price)}
          </span>
        </div>

        <div className="w-full relative pt-2 flex gap-1.5 items-center">
          {isAddedSuccess && (
            <span className="absolute top-[-10px] left-0 right-0 text-center text-[9px] font-black text-emerald-600 animate-pulse z-20 bg-white/90 py-0.5">
              Added to Cart!
            </span>
          )}

          <button
            disabled={isAddedSuccess || isMutating}
            onClick={handleDirectAddToCart}
            className={`flex-1 text-center font-extrabold rounded-xl shadow-2xs transition duration-200 cursor-pointer z-10 whitespace-nowrap truncate text-[10px] sm:text-xs px-2.5 py-2 ${
              isAddedSuccess
                ? 'bg-emerald-600 text-white font-black cursor-default'
                : 'bg-rose-900 hover:bg-rose-800 text-white active:scale-95'
            } ${isMutating && !isAddedSuccess ? 'opacity-50 cursor-wait' : ''}`}
          >
            {isAddedSuccess ? ''  Added' : isMutating ? 'Updating...' : 'Add to Cart'}
          </button>

          <button
            onClick={handleCardRedirection}
            className="bg-gray-100 hover:bg-gray-200 text-rose-950 font-extrabold rounded-xl px-2.5 py-2 text-[10px] sm:text-xs transition active:scale-95 cursor-pointer z-10"
          >
            Details
          </button>
        </div>
      </div>
    </div>
  );
});

export default ProductCard;
```

File: src/components/ProductGrid.tsx

```tsx
import ProductCard from './ProductCard';
import type { Product } from '../types/database';

interface ProductGridProps {
  products: Product[];
  loading: boolean;
}

export default function ProductGrid({ products, loading }: ProductGridProps) {
  if (loading) {
    return (
      <div className="py-20 text-center flex flex-col items-center justify-center gap-3">
        <div className="w-8 h-8 border-4 border-rose-800 border-t-transparent rounded-full animate-spin"></
div>
        <p className="text-gray-400 text-xs font-bold uppercase tracking-wider">Syncing Saree Showcase...</p>
      </div>
    );
  }

  if (products.length === 0) {
    return (
      <div className="py-16 text-center border border-dashed border-rose-200 bg-white rounded-3xl mx-2">
        <p className="text-gray-400 font-semibold text-sm">No saree inventory items match your selection.</p>
      </div>
    );
  }

  return (
    <div className="grid grid-cols-2 sm:grid-cols-3 lg:grid-cols-4 min-[2000px]:grid-cols-5 gap-3 sm:gap-6 w-
full px-1">  {products.map((item) => (
        <div key={item.id} className="min-w-0 overflow-hidden">
          <ProductCard
            id={item.id}
            title={item.title}
            price={item.price}
            image_url={item.image_url}
            category={item.category}
            description={item.description}
          />
        </div>
      ))}
    </div>
  );
}
```

File: src/components/FeaturedLanes.tsx

```tsx
import { Link } from 'react-router-dom';
import ProductCard from './ProductCard';
import type { Product } from '../types/database';

interface FeaturedLanesProps {
  products: Product[];
  categories: string[];
  limitProducts: number;
}

export default function FeaturedLanes({ products, categories, limitProducts }: FeaturedLanesProps) {
  const activeCategories = categories.filter((cat) => cat !== 'All');

  const lanesToRender = activeCategories
    .map((catName) => {
      const matchingItems = products.filter(
        (p) => p.category && p.category.toLowerCase() === catName.toLowerCase()
      );

      return {
        categoryName: catName,
        products: matchingItems.slice(0, Math.max(4, limitProducts)),
      };
    })
    .filter((lane) => lane.products.length > 0);

  if (lanesToRender.length === 0) {
    return (
      <div className="py-16 text-center border border-dashed border-rose-200 bg-white rounded-3xl mx-2">
        <p className="text-gray-400 font-bold text-xs tracking-wider uppercase">
          No saree collections available in this view.
        </p>
      </div>
    );
  }

  return (
    <div className="space-y-12 w-full">
      {lanesToRender.map((lane) => (
        <div key={lane.categoryName} className="space-y-4">

          {/* HEADER BLOCK */}
          <div className="flex items-center justify-between border-b border-rose-100 pb-2.5 gap-2 px-1">
            <div className="flex items-center gap-2 min-w-0">
              <h3 className="text-base sm:text-xl font-serif font-bold text-rose-950 tracking-tight uppercase truncate">
                {lane.categoryName} Collection
              </h3>
              <span className="text-[9px] sm:text-[10px] font-black bg-rose-100 text-rose-900 px-2 py-0.5 rounded-md uppercase tracking-wider whitespace-nowrap shrink-0">
                Signature Selection
              </span>
            </div>

            <Link
              to={`/category/${encodeURIComponent(lane.categoryName.toLowerCase())}`}
              className="text-[10px] sm:text-xs font-black bg-rose-900 text-white hover:bg-rose-800 px-3 py-1.5 rounded-xl shadow-2xs transition duration-150 active:scale-95 flex items-center gap-1 whitespace-nowrap shrink-0 cursor-pointer group"
            >
              <span>View All</span>
              <span className="transition-transform duration-150 group-hover:translate-x-0.5">!’</span>
            </Link>
          </div>

          {/* RESPONSIVE ROW GRID */}
          <div className="grid grid-cols-2 sm:grid-cols-3 lg:grid-cols-4 min-[2000px]:grid-cols-5 gap-3 sm:gap-6 w-full">
            {lane.products.map((item, index) => (
              <div
                key={item.id}
                className={`min-w-0 overflow-hidden ${
                  index === 3 ? 'hidden lg:block' : index === 4 ? 'hidden min-[2000px]:block' : 'block'
                }`}
              >
                <ProductCard
                  id={item.id}
                  title={item.title}
                  price={item.price}
                  image_url={item.image_url}
                  category={item.category}
```

File: src/components/FeaturedLanes.tsx (continued)

```
77 |                   description={item.description}
78 |                 />
79 |               </div>
80 |             ))}
81 |           </div>
82 |
83 |         </div>
84 |       ))}
85 |     </div>
86 |   );
87 | }
```

File: src/components/CheckoutForm.tsx

```tsx
import React, { useState } from 'react';
import { supabase } from '../supabaseClient';
import { formatBDT } from '../types/database';
import { getErrorMessage } from '../utils/errorHandling';

interface CartItemRecord {
  id: number;
  quantity: number;
  products: {
    id: number;
    title: string;
    price: number;
    image_url: string;
  };
}

interface CheckoutFormProps {
  cartItems: CartItemRecord[];
  subtotal: number;
  shipping?: number;
  grandTotal?: number;
  onClose?: () => void;
  onOrderSuccess: () => void;
}

export default function CheckoutForm({
  cartItems,
  subtotal,
  shipping = 150,
  grandTotal: passedGrandTotal,
  onClose,
  onOrderSuccess,
}: CheckoutFormProps) {
  const computedGrandTotal = passedGrandTotal ?? (subtotal + (subtotal > 0 ? shipping : 0));

  const [loading, setLoading] = useState<boolean>(false);
  const [fullName, setFullName] = useState<string>('');
  const [phone, setPhone] = useState<string>('');
  const [errorMsg, setErrorMsg] = useState<string | null>(null);

  // Address configuration
  const [addressType, setAddressType] = useState<'Standard' | 'University'>('Standard');
  const [district, setDistrict] = useState<string>('');
  const [upazila, setUpazila] = useState<string>('');
  const [villageArea, setVillageArea] = useState<string>('');
  const [universityName, setUniversityName] = useState<string>('');
  const [hallName, setHallName] = useState<string>('');

  // Payment states
  const [paymentMethod, setPaymentMethod] = useState<'COD' | 'bKash' | 'Nagad'>('COD');
  const [transactionId, setTransactionId] = useState<string>('');

  // Strict Bangladeshi 11-digit mobile validation regex
  const validateBDPhone = (num: string) => {
    const cleaned = num.replace(/\D/g, '');
    return /^01[3-9]\d{8}$/.test(cleaned);
  };

  const handleSubmitOrder = async (e: React.FormEvent) => {
    e.preventDefault();
    setErrorMsg(null);

    const sanitizedPhone = phone.replace(/\D/g, '');

    if (!fullName.trim()) {
      setErrorMsg('Please enter your Full Customer Name.');
      return;
    }

    if (!sanitizedPhone || !validateBDPhone(sanitizedPhone)) {
      setErrorMsg('Please enter a valid 11-digit Bangladeshi mobile number starting with 01 (e.g., 01712345678).');
      return;
    }

    if (addressType === 'Standard' && (!district.trim() || !upazila.trim() || !villageArea.trim())) {
      setErrorMsg('Please fill in all home delivery address fields (District, Upazila, and Street/House).');
```

File: src/components/CheckoutForm.tsx (continued)

```
      return;
    }

    if (addressType === 'University' && (!universityName.trim() || !hallName.trim())) {
      setErrorMsg('Please state your University Name and designated Hall/Building.');
      return;
    }

    if ((paymentMethod === 'bKash' || paymentMethod === 'Nagad') && !transactionId.trim()) {
      setErrorMsg(`Please enter your ${paymentMethod} payment Verification Transaction ID (TrxID).`);
      return;
    }

    setLoading(true);

    try {
      const { data: { user } } = await supabase.auth.getUser();
      if (!user) throw new Error('Authentication expired. Please sign in to place your order.');

      // Compile structured destination payload
      const shippingDestination = addressType === 'Standard'
        ? { type: 'Standard', district: district.trim(), upazila: upazila.trim(), villageArea:
villageArea.trim() }
        : { type: 'University', universityName: universityName.trim(), hallName: hallName.trim() };

      const fullAddressSummary = addressType === 'Standard'
        ? `Area: ${villageArea.trim()}, Upazila: ${upazila.trim()}, District: ${district.trim()}`
        : `Hall: ${hallName.trim()}, Campus: ${universityName.trim()}`;

      // 1. Insert core order log
      const { data: orderData, error: orderError } = await supabase
        .from('orders')
        .insert([
          {
            user_id: user.id,
            total_amount: computedGrandTotal,
            shipping_address: fullAddressSummary,
            status: 'Pending',
            contact_number: sanitizedPhone,
            shipping_destination: shippingDestination,
            payment_details: {
              method: paymentMethod,
              trx_id: paymentMethod !== 'COD' ? transactionId.trim().toUpperCase() : 'CASH_ON_DELIVERY',
            },
          },
        ])
        .select();

      if (orderError) throw orderError;
      const newOrder = (orderData as any[])[0];

      // 2. Insert itemized order line breakdowns
      const orderItemsPayload = cartItems.map((item) => ({
        order_id: newOrder.id,
        product_id: item.products.id,
        quantity: item.quantity,
        price_at_purchase: item.products.price,
      }));

      const { error: itemsError } = await supabase.from('order_items').insert(orderItemsPayload);
      if (itemsError) throw itemsError;

      // 3. Clear active user shopping cart
      await supabase.from('cart_items').delete().eq('user_id', user.id);

      onOrderSuccess();
    } catch (err: unknown) {
      const msg = getErrorMessage(err);
      setErrorMsg('Order compilation failed: ' + msg);
    } finally {
      setLoading(false);
    }
  };

  return (
    <div className="w-full bg-white border border-rose-100 p-5 sm:p-7 rounded-3xl shadow-xl font-sans text-
gray-800">

      {/* Header Bar */}
      <div className="flex items-center justify-between border-b border-rose-100 pb-3 mb-4">
```

File: src/components/CheckoutForm.tsx (continued)

```
        <div>
          <h2 className="text-base sm:text-lg font-black text-rose-950 tracking-tight flex items-center gap-2">
            <span>Ø=Üæ</span> Courier Delivery & Payment Details
          </h2>
          <p className="text-[11px] text-gray-500 font-medium mt-0.5">
            Complete your shipping location and select local payment options.
          </p>
        </div>
        {onClose && (
          <button
            type="button"
            onClick={onClose}
            className="p-1.5 rounded-xl bg-rose-50 text-rose-700 hover:bg-rose-100 transition cursor-pointer"
            title="Close Checkout"
          >
            '
          </button>
        )}
      </div>

      {errorMsg && (
        <div className="mb-4 p-3 bg-red-50 border border-red-200 text-red-600 rounded-xl text-xs font-bold flex items-start gap-2">
          <span>& þ </span>
          <span>{errorMsg}</span>
        </div>
      )}

      {/* Payable Amount Summary Banner */}
      <div className="bg-rose-50/70 border border-rose-100 p-3 rounded-2xl flex items-center justify-between mb-4">
        <span className="text-xs font-bold text-rose-900">Total Payable Amount:</span>
        <span className="text-base sm:text-lg font-black text-rose-700 font-mono">
          {formatBDT(computedGrandTotal)}
        </span>
      </div>

      <form onSubmit={handleSubmitOrder} className="space-y-4">

        {/* Customer Information */}
        <div className="grid grid-cols-1 sm:grid-cols-2 gap-3.5">
          <div>
            <label className="block text-[10px] font-bold text-gray-500 uppercase tracking-wider mb-1">
              Customer Full Name *
            </label>
            <input
              type="text"
              required
              value={fullName}
              onChange={(e) => setFullName(e.target.value)}
              placeholder="e.g., Nusrat Jahan"
              className="w-full text-xs border border-gray-200 bg-gray-50/50 p-2.5 rounded-xl focus:bg-white focus:outline-rose-500 font-medium"
            />
          </div>
          <div>
            <label className="block text-[10px] font-bold text-gray-500 uppercase tracking-wider mb-1">
              Active BD Mobile (11 Digits) *
            </label>
            <input
              type="tel"
              inputMode="numeric"
              maxLength={11}
              required
              value={phone}
              onChange={(e) => setPhone(e.target.value.replace(/\D/g, ''))}
              placeholder="e.g., 01712345678"
              className="w-full text-xs border border-gray-200 bg-gray-50/50 p-2.5 rounded-xl focus:bg-white focus:outline-rose-500 font-mono font-bold"
            />
          </div>
        </div>

        {/* Address Type Selector */}
        <div>
          <label className="block text-[10px] font-bold text-gray-500 uppercase tracking-wider mb-1.5">
            Delivery Channel Type *
          </label>
          <div className="grid grid-cols-2 gap-2 p-1 bg-gray-100 rounded-xl">
            <button
              type="button"
              onClick={() => setAddressType('Standard')}
```

File: src/components/CheckoutForm.tsx (continued)

```
233 |                 className={`py-2 text-[11px] font-bold rounded-lg transition-all cursor-pointer ${
234 |                   addressType === 'Standard' ? 'bg-white text-rose-900 shadow-xs' : 'text-gray-500'
235 |                 }`}
236 |               >
237 |                 Ø<ßà Home Delivery
238 |               </button>
239 |               <button
240 |                 type="button"
241 |                 onClick={() => setAddressType('University')}
242 |                 className={`py-2 text-[11px] font-bold rounded-lg transition-all cursor-pointer ${
243 |                   addressType === 'University' ? 'bg-white text-purple-900 shadow-xs' : 'text-gray-500'
244 |                 }`}
245 |               >
246 |                 Ø<ß“ Varsity / Campus Hall
247 |               </button>
248 |             </div>
249 |           </div>
250 |
251 |           {/* Conditional Address Fields */}
252 |           {addressType === 'Standard' ? (
253 |             <div className="space-y-3 p-3.5 bg-rose-50/30 rounded-2xl border border-rose-100">
254 |               <div className="grid grid-cols-2 gap-3">
255 |                 <div>
256 |                   <label className="block text-[9px] font-bold text-gray-500 uppercase mb-1">District / Zila
*</label>
257 |                   <input
258 |                     type="text"
259 |                     required={addressType === 'Standard'}
260 |                     value={district}
261 |                     onChange={(e) => setDistrict(e.target.value)}
262 |                     placeholder="e.g., Dhaka"
263 |                     className="w-full text-xs border border-gray-200 bg-white p-2.5 rounded-xl focus:outline-
rose-500 font-medium"  />
264 |
265 |                 </div>
266 |                 <div>
267 |                   <label className="block text-[9px] font-bold text-gray-500 uppercase mb-1">Upazila / Thana
*</label>
268 |                   <input
269 |                     type="text"
270 |                     required={addressType === 'Standard'}
271 |                     value={upazila}
272 |                     onChange={(e) => setUpazila(e.target.value)}
273 |                     placeholder="e.g., Dhanmondi"
274 |                     className="w-full text-xs border border-gray-200 bg-white p-2.5 rounded-xl focus:outline-
rose-500 font-medium"  />
275 |
276 |                 </div>
277 |               </div>
278 |               <div>
279 |                 <label className="block text-[9px] font-bold text-gray-500 uppercase mb-1">
280 |                   House, Road, Area Details *
281 |                 </label>
282 |                 <input
283 |                   type="text"
284 |                   required={addressType === 'Standard'}
285 |                   value={villageArea}
286 |                   onChange={(e) => setVillageArea(e.target.value)}
287 |                   placeholder="e.g., House #24, Road #7, Sector 4"
288 |                   className="w-full text-xs border border-gray-200 bg-white p-2.5 rounded-xl focus:outline-
rose-500 font-medium"/>
289 |
290 |               </div>
291 |             </div>
292 |           ) : (
293 |             <div className="space-y-3 p-3.5 bg-purple-50/30 rounded-2xl border border-purple-100">
294 |               <div className="grid grid-cols-1 sm:grid-cols-2 gap-3">
295 |                 <div>
296 |                   <label className="block text-[9px] font-bold text-gray-500 uppercase mb-1">University Name
*</label>
297 |                   <input
298 |                     type="text"
299 |                     required={addressType === 'University'}
300 |                     value={universityName}
301 |                     onChange={(e) => setUniversityName(e.target.value)}
302 |                     placeholder="e.g., Dhaka University"
303 |                     className="w-full text-xs border border-gray-200 bg-white p-2.5 rounded-xl focus:outline-
purple-500 font-medium"/>
304 |
305 |                 </div>
306 |                 <div>
307 |                   <label className="block text-[9px] font-bold text-gray-500 uppercase mb-1">Hall / Department
Lounge *</label>
308 |                   <input
309 |                     type="text"
310 |                     required={addressType === 'University'}
```

File: src/components/CheckoutForm.tsx (continued)

```
311 |                   value={hallName}
312 |                   onChange={(e) => setHallName(e.target.value)}
313 |                   placeholder="e.g., Ruqayyah Hall, Room 302"
314 |                   className="w-full text-xs border border-gray-200 bg-white p-2.5 rounded-xl focus:outline-
purple-500 font-medium"/>
315 |               </div>
316 |             </div>
317 |           </div>
318 |         )}
319 |
320 |         {/* Local Payment Method Selection */}
321 |         <div>
322 |           <label className="block text-[10px] font-bold text-gray-500 uppercase tracking-wider mb-2">
323 |             Select Payment Channel *
324 |           </label>
325 |           <div className="grid grid-cols-3 gap-2">
326 |             <button
327 |               type="button"
328 |               onClick={() => { setPaymentMethod('COD'); setTransactionId(''); }}
329 |               className={`p-2.5 border rounded-2xl text-center flex flex-col items-center justify-center
transition-all cursor-pointer ${
330 |                 paymentMethod === 'COD'
331 |                   ? 'border-emerald-600 bg-emerald-50 text-emerald-800 font-bold'
332 |                   : 'border-gray-200 text-gray-600 hover:border-gray-300'
333 |               }`}
334 |             >
335 |               <span className="text-xl mb-0.5">Ø=Üµ</span>
336 |               <span className="text-[10px] font-black tracking-tight leading-none">Cash On Delivery</span>
337 |             </button>
338 |
339 |             <button
340 |               type="button"
341 |               onClick={() => setPaymentMethod('bKash')}
342 |               className={`p-2.5 border rounded-2xl text-center flex flex-col items-center justify-center
transition-all cursor-pointer ${
343 |                 paymentMethod === 'bKash'
344 |                   ? 'border-pink-600 bg-pink-50 text-pink-800 font-bold'
345 |                   : 'border-gray-200 text-gray-600 hover:border-gray-300'
346 |               }`}
347 |             >
348 |               <span className="text-xl mb-0.5">Ø<ß8</span>
349 |               <span className="text-[10px] font-black tracking-tight leading-none">bKash (01648038036)</span>
350 |             </button>
351 |
352 |             <button
353 |               type="button"
354 |               onClick={() => setPaymentMethod('Nagad')}
355 |               className={`p-2.5 border rounded-2xl text-center flex flex-col items-center justify-center
transition-all cursor-pointer ${
356 |                 paymentMethod === 'Nagad'
357 |                   ? 'border-orange-600 bg-orange-50 text-orange-800 font-bold'
358 |                   : 'border-gray-200 text-gray-600 hover:border-gray-300'
359 |               }`}
360 |             >
361 |               <span className="text-xl mb-0.5">Ø=Ý%</span>
362 |               <span className="text-[10px] font-black tracking-tight leading-none">Nagad (01648038036)</span>
363 |             </button>
364 |           </div>
365 |         </div>
366 |
367 |         {/* Transaction Reference Panel for Mobile Wallets */}
368 |         {paymentMethod !== 'COD' && (
369 |           <div className="bg-amber-50 border border-amber-200 p-3.5 rounded-2xl space-y-2">
370 |             <div className="text-[11px] text-amber-900 font-medium leading-relaxed">
371 |               Ø=ÜI Please Send Money <span className="font-extrabold">{formatBDT(computedGrandTotal)}</span> to
our merchant wallet number: <span className="font-black underline font-mono">01648038036</span>. After payment,
enter your TrxID below:
372 |             </div>
373 |
374 |             <div>
375 |               <label className="block text-[9px] font-bold text-amber-800 uppercase mb-1">
376 |                 Payment Verification Transaction ID (TrxID) *
377 |               </label>
378 |               <input
379 |                 type="text"
380 |                 required
381 |                 value={transactionId}
382 |                 onChange={(e) => setTransactionId(e.target.value)}
383 |                 placeholder="e.g., BK92837482"
384 |                 className="w-full text-xs border border-amber-300 bg-white p-2.5 rounded-xl focus:outline-
amber-600 font-mono tracking-wider font-bold uppercase"
385 |               />
386 |             </div>
387 |           </div>
388 |         )}
```

File: src/components/CheckoutForm.tsx (continued)

```
389 |
390 |         <button
391 |           type="submit"
392 |           disabled={loading}
393 |           className="w-full text-xs font-black uppercase tracking-wider bg-rose-900 hover:bg-rose-800 text-
white py-3.5 rounded-2xl shadow-md transition disabled:bg-gray-200 cursor-pointer mt-2"
394 |
395 |           {loading ? 'Processing Order...' : `Confirm Saree Order (${formatBDT(computedGrandTotal)})`}
396 |         </button>
397 |       </form>
398 |     </div>
399 |   );
400 | }
```

File: src/components/SearchBar.tsx

```tsx
import React, { useState, useEffect, useRef } from 'react';
import { supabase } from '../supabaseClient';
import { getErrorMessage } from '../utils/errorHandling';

interface SearchBarProps {
  onSearch: (query: string) => void;
  isLoading: boolean;
}

export default function SearchBar({ onSearch, isLoading }: SearchBarProps) {
  const [inputValue, setInputValue] = useState<string>('');
  const [suggestions, setSuggestions] = useState<string[]>([]);
  const [showDropdown, setShowDropdown] = useState<boolean>(false);
  const dropdownRef = useRef<HTMLDivElement>(null);

  useEffect(() => {
    const fetchSuggestions = async () => {
      const trimmed = inputValue.trim();
      if (trimmed.length < 1) {
        setSuggestions([]);
        return;
      }
      try {
        const { data, error } = await supabase
          .from('products')
          .select('title')
          .ilike('title', `%${trimmed}%`)
          .limit(10);

        if (!error && data) {
          const uniqueTitles = Array.from(new Set(data.map((p) => p.title)));
          const sortedTitles = uniqueTitles.sort((a, b) => {
            const indexA = a.toLowerCase().indexOf(trimmed.toLowerCase());
            const indexB = b.toLowerCase().indexOf(trimmed.toLowerCase());
            return indexA - indexB;
          });

          setSuggestions(sortedTitles.slice(0, 6));
        }
      } catch (err) {
        console.error('Error fetching saree search suggestions:', getErrorMessage(err));
      }
    };

    const debounce = setTimeout(() => {
      fetchSuggestions();
    }, 150);

    return () => clearTimeout(debounce);
  }, [inputValue]);

  useEffect(() => {
    function handleClickOutside(event: MouseEvent) {
      if (dropdownRef.current && !dropdownRef.current.contains(event.target as Node)) {
        setShowDropdown(false);
      }
    }
    document.addEventListener('mousedown', handleClickOutside);
    return () => document.removeEventListener('mousedown', handleClickOutside);
  }, []);

  const handleSearchTrigger = (queryToSend: string) => {
    onSearch(queryToSend);
    setShowDropdown(false);
  };

  const handleKeyDown = (e: React.KeyboardEvent<HTMLInputElement>) => {
    if (e.key === 'Enter') {
      handleSearchTrigger(inputValue);
    }
  };

  const handleClearSearch = () => {
    setInputValue('');
    onSearch('');
    setSuggestions([]);
```

File: src/components/SearchBar.tsx (continued)

```
    setShowDropdown(false);
  };

  const renderHighlightedText = (text: string, highlight: string) => {
    if (!highlight.trim()) return <span>{text}</span>;
    const parts = text.split(new RegExp(`(${highlight.replace(/[-\/\\^$*+?.()|[\]{}]/g, '\\$&')})`, 'gi'));
    return (
      <span>
        {parts.map((part, i) =>
          part.toLowerCase() === highlight.toLowerCase() ? (
            <b key={i} className="text-rose-950 font-black">
              {part}
            </b>
          ) : (
            <span key={i} className="text-gray-600 font-medium">
              {part}
            </span>
          )
        )}
      </span>
    );
  };

  return (
    <div className="w-full max-w-3xl mx-auto my-4 px-2 box-border relative z-40" ref={dropdownRef}>
      <div classNam="flex items-center bg-white border-2 border-rose-200 rounded-2xl overflow-hidden shadow-xs hover:border-rose-300 focus-within:ring-4 focus-within:ring-rose-500/20 focus-within:border-rose-700 transition-all duration-150">
        <input
          type="text"
          value={inputValue}
          onFocus={() => setShowDropdown(true)}
          placeholder="Search Menakkhi Sarees (Katan, Jamdani, Silk)..."
          onChange={(e) => {
            setInputValue(e.target.value);
            setShowDropdown(true);
          }}
          onKeyDown={handleKeyDown}
          className="w-full pl-4 pr-10 py-3 bg-transparent font-sans text-rose-950 placeholder-gray-400 text-xs sm:text-sm font-bold outline-none border-none"
        />

        {inputValue.length > 0 && !isLoading && (
          <button
            type="button"
            onClick={handleClearSearch}
            className="absolute right-3 bg-gray-100 hover:bg-rose-100 text-rose-950 font-black text-xs transition rounded-full flex items-center justify-center w-6 h-6 cursor-pointer"
            title="Clear search"
          >
            '
          </button>
        )}
      </div>

      <button
        type="button"
        onClick={() => handleSearchTrigger(inputValue)}
        className="bg-rose-900 hover:bg-rose-800 text-amber-200 self-stretch px-6 flex items-center justify-center cursor-pointer transition shrink-0 font-bold text-xs uppercase tracking-wider"
        title="Submit Search"
      >
        {isLoading ? (
          <div className="w-4 h-4 border-2 border-amber-200 border-t-transparent rounded-full animate-spin"></div>
        ) : (
          <svg
            xmlns="http://www.w3.org/2000/svg"
            className="h-5 w-5"
            fill="none"
            viewBox="0 0 24 24"
            stroke="currentColor"
            strokeWidth={2.5}
          >
            <path strokeLinecap="round" strokeLinejoin="round" d="M21 21l-5.197-5.197m0 0A7.5 7.5 0 105.196 5.196a7.5 7.5 0 0010.607 10.607z" />
          </svg>
        )}
      </button>
    </div>

    {showDropdown && suggestions.length > 0 && (
      <div className="absolute left-1 right-1 mt-2 bg-white border border-rose-100 shadow-2xl rounded-2xl z-50 overflow-hidden max-h-64 overflow-y-auto">
        {suggestions.map((suggestion, index) => (
```

File: src/components/SearchBar.tsx (continued)

```
155 |             <div
156 |               key={index}
157 |               onClick={() => {
158 |                 setInputValue(suggestion);
159 |                 handleSearchTrigger(suggestion);
160 |               }}
161 |               className="px-4 py-3 text-xs sm:text-sm hover:bg-rose-50 cursor-pointer transition flex items-
center gap-3 border-b border-rose-50 last:border-none"
162 |             >
163 |               <svg className="w-4 h-4 text-rose-400 shrink-0" fill="none" stroke="currentColor"
strokeWidth={2.5} viewBox="0 0 24 24">
164 |                 <path strokeLinecap="round" strokeLinejoin="round" d="M21 21l-6-6m2-5a7 7 0 11-14 0 7 7 0
0114 0z" />
165 |               </svg>
166 |               <span className="truncate">{renderHighlightedText(suggestion, inputValue)}</span>
167 |             </div>
168 |           ))}
169 |         </div>
170 |       )}
171 |     </div>
172 |   );
173 | }
```

File: src/components/Footer.tsx

```tsx
import { Link } from 'react-router-dom';

export default function Footer() {
  const currentYear = new Date().getFullYear();

  return (
    <footer className="bg-rose-950 text-rose-200/80 pt-12 pb-6 px-6 font-sans mt-auto border-t border-rose-900">
      <div className="max-w-6xl mx-auto grid grid-cols-1 md:grid-cols-3 gap-8 mb-8">

        {/* Brand Info */}
        <div>
          <div className="flex items-center gap-2 mb-3">
            <span className="text-xl">Ø<ß8</span>
            <h3 className="text-white text-lg font-serif font-bold tracking-tight">Menakkhi Sarees</h3>
          </div>
          <p className="text-xs sm:text-sm leading-relaxed text-rose-200/70">
            Your premier online destination for authentic Dhakai Jamdani, Katan Silk, Rajshahi Silk, Muslin, Organza, and bespoke Bridal Sarees. Handcrafted with traditional Bangladeshi heritage craftsmanship.
          </p>
        </div>

        {/* Quick Links */}
        <div>
          <h4 className="text-white text-sm font-black uppercase tracking-wider mb-4">Saree Collections</h4>
          <ul className="space-y-2 text-xs font-semibold">
            <li><Link to="/products" className="hover:text-amber-300 transition">All Saree Weaves</Link></li>
            <li><Link to="/category/katan" className="hover:text-amber-300 transition">Pure Katan Silk</Link></li>
            <li><Link to="/category/jamdani" className="hover:text-amber-300 transition">Dhakai Muslin Jamdani</Link></li>
            <li><Link to="/category/bridal%20collection" className="hover:text-amber-300 transition">Royal Bridal Collection</Link></li>
          </ul>
        </div>

        {/* Customer Center */}
        <div>
          <h4 className="text-white text-sm font-black uppercase tracking-wider mb-4">Customer Care</h4>
          <ul className="space-y-2 text-xs font-semibold">
            <li><Link to="/cart" className="hover:text-amber-300 transition">Shopping Bag & Checkout</Link></li>
            <li><Link to="/profile" className="hover:text-amber-300 transition">Order History & Vouchers</Link></li>
            <li><Link to="/orders" className="hover:text-amber-300 transition">Top Sales Showcase</Link></li>
          </ul>
        </div>

      </div>

      <div className="border-t border-rose-900/80 max-w-6xl mx-auto pt-6 text-center text-xs text-rose-300/50 flex flex-col sm:flex-row justify-between items-center gap-2">
        <div>
          &copy; {currentYear} Menakkhi Sarees Boutique. All rights reserved.
        </div>
        <div className="text-[10px] font-mono text-amber-300/80">
          Delivery Across Bangladesh (bKash | Nagad | Cash on Delivery)
        </div>
      </div>
    </footer>
  );
}
```

File: src/components/Subscribe.tsx

```tsx
import { useState, useEffect } from 'react';
import { useNavigate } from 'react-router-dom';
import { supabase } from '../supabaseClient';
import { getErrorMessage } from '../utils/errorHandling';

export default function Subscribe() {
  const navigate = useNavigate();
  const [isSubscribed, setIsSubscribed] = useState<boolean>(false);
  const [loading, setLoading] = useState<boolean>(true);
  const [actionLoading, setActionLoading] = useState<boolean>(false);

  useEffect(() => {
    async function checkSubscriptionStatus() {
      try {
        const { data: { user } } = await supabase.auth.getUser();
        if (user && user.email) {
          const { data, error } = await supabase
            .from('subscribers')
            .select('email')
            .eq('email', user.email.toLowerCase())
            .maybeSingle();

          if (!error && data) {
            setIsSubscribed(true);
          }
        }
      } catch (err) {
        console.error('Subscription check error:', getErrorMessage(err));
      } finally {
        setLoading(false);
      }
    }
    checkSubscriptionStatus();
  }, []);

  const handleSubscribeAction = async () => {
    try {
      const { data: { user } } = await supabase.auth.getUser();

      if (!user || !user.email) {
        navigate('/login');
        return;
      }

      setActionLoading(true);

      const { error } = await supabase
        .from('subscribers')
        .insert([{ email: user.email.toLowerCase() }]);

      if (error) {
        if (error.code === '23505') {
          setIsSubscribed(true);
          return;
        }
        throw error;
      }

      setIsSubscribed(true);
    } catch (err) {
      console.error('Failed to subscribe:', getErrorMessage(err));
      alert('Could not update subscription settings.');
    } finally {
      setActionLoading(false);
    }
  };

  const handleUnsubscribeAction = async () => {
    try {
      const { data: { user } } = await supabase.auth.getUser();
      if (!user || !user.email) return;

      setActionLoading(true);

      const { error } = await supabase
        .from('subscribers')
```

File: src/components/Subscribe.tsx (continued)

```
        .delete()
        .eq('email', user.email.toLowerCase());

      if (error) throw error;

      setIsSubscribed(false);
    } catch (err) {
      console.error('Failed to unsubscribe:', getErrorMessage(err));
      alert('Could not update subscription settings.');
    } finally {
      setActionLoading(false);
    }
  };

  if (loading) {
    return (
      <section className="my-12 bg-rose-900 rounded-3xl p-8 text-center text-white shadow-md animate-pulse">
        <div className="h-6 w-48 bg-rose-800 rounded mx-auto mb-3"></div>
        <div className="h-4 w-64 bg-rose-800 rounded mx-auto"></div>
      </section>
    );
  }

  return (
    <section className="my-12 bg-gradient-to-br from-rose-950 via-rose-900 to-red-950 rounded-3xl p-8 md:p-12 text-center text-white shadow-xl relative overflow-hidden">
      <div className="absolute -top-12 -right-12 w-40 h-40 bg-amber-400/10 rounded-full blur-2xl pointer-events-none"></div>
      <div className="absolute -bottom-12 -left-12 w-40 h-40 bg-rose-500/20 rounded-full blur-2xl pointer-events-none"></div>

      <div className="max-w-xl mx-auto relative z-10">
        <span className="inline-block text-3xl mb-2 select-none">
          {isSubscribed ? 'Ø=Ý ' : 'Ø=ÜŒ'}
        </span>
        <h2 className="text-2xl md:text-3xl font-serif font-bold mb-3 tracking-tight text-amber-100">
          {isSubscribed ? 'You are Subscribed!' : 'Menakkhi VIP Saree Club'}
        </h2>
        <p className="text-rose-100/80 text-xs md:text-sm mb-6 max-w-md mx-auto leading-relaxed">
          {isSubscribed
            ? 'You are currently registered for new saree weave drops, festival pre-orders, and VIP discount alerts.'
            : 'Subscribe to receive early notifications on exclusive Jamdani, Silk, and Katan drops, plus festive offers.'}
        </p>

        <div className="flex flex-col items-center justify-center max-w-xs mx-auto">
          {!isSubscribed ? (
            <button
              disabled={actionLoading}
              onClick={handleSubscribeAction}
              className="w-full bg-amber-400 hover:bg-amber-300 text-rose-950 font-black text-xs uppercase tracking-wider py-3.5 px-8 rounded-2xl shadow-lg transition-all duration-200 active:scale-95 cursor-pointer disabled:opacity-50"
            >
              {actionLoading ? 'Processing...' : 'Subscribe to Saree Drops'}
            </button>
          ) : (
            <div className="w-full space-y-3">
              <div className="w-full bg-emerald-600 text-white font-black text-xs uppercase tracking-widest py-3.5 px-8 rounded-2xl shadow-md border border-emerald-500/30 select-none">
                VIP Member Subscribed
              </div>

              <button
                disabled={actionLoading}
                onClick={handleUnsubscribeAction}
                className="text-[11px] font-bold text-rose-200 hover:text-white transition underline cursor-pointer disabled:opacity-40"
              >
                {actionLoading ? 'Updating...' : 'Unsubscribe from notifications'}
              </button>
            </div>
          )}
        </div>
      </div>
    </section>
  );
}
```

File: src/components/ProtectedRoute.tsx

```
import React, { useEffect, useState } from 'react';Ð
import { Navigate } from 'react-router-dom';Ð
import { supabase } from '../supabaseClient';Ð
Ð
interface ProtectedRouteProps {Ð
  children: React.ReactElement;Ð
  requireAdmin?: boolean;Ð
}Ð
Ð
export default function ProtectedRoute({ children, requireAdmin = false }: ProtectedRouteProps) {Ð
  const [isAuthenticated, setIsAuthenticated] = useState<boolean | null>(null);Ð
  const [isAdminUser, setIsAdminUser] = useState<boolean | null>(null); // Ø=Þ€ Changed to null to track loading state accurately
  const [loading, setLoading] = useState<boolean>(true); // Ø=Þ€ Explicit layout lifecycle flagÐ
Ð
  useEffect(() => {Ð
    const checkAuth = async () => {Ð
      try {Ð
        setLoading(true);Ð
        const { data: { user } } = await supabase.auth.getUser();Ð
        Ð
        if (!user) {Ð
          setIsAuthenticated(false);Ð
          setIsAdminUser(false);Ð
          return;Ð
        }Ð
Ð
        setIsAuthenticated(true);Ð
Ð
        // If the path requires administrative clearanceÐ
        if (requireAdmin) {Ð
          // Check user metadata or verify through your specific validation criteriaÐ
          const isUserAdmin = user.user_metadata?.role === 'admin' || user.email?.endsWith('@admin.com');Ð
          setIsAdminUser(!!isUserAdmin);Ð
        } else {Ð
          setIsAdminUser(false);Ð
        }Ð
      } catch (err) {Ð
        setIsAuthenticated(false);Ð
        setIsAdminUser(false);Ð
      } finally {Ð
        setLoading(false); // Ø=Ý  Only turn off loading flag once all states are resolved!Ð
      }Ð
    };Ð
Ð
    checkAuth();Ð
  }, [requireAdmin]);Ð
Ð
  // #ó Hold structural renders open until the auth metadata pipeline finishes executing completelyÐ
  if (loading || isAuthenticated === null || (requireAdmin && isAdminUser === null)) {Ð
    return (Ð
      <div className="min-h-screen bg-slate-50 flex items-center justify-center font-sans">Ð
        <div className="flex flex-col items-center gap-3">Ð
          <div className="w-6 h-6 border-2 border-blue-600 border-t-transparent rounded-full animate-spin"></div>Ð
          <span className="text-xs font-bold text-gray-400 tracking-wide animate-pulse">Verifying Security Clearance...</span>Ð
        </div>Ð
      </div>Ð
    );Ð
  }Ð
Ð
  // Ø=Þ¨ 1. Unauthenticated users get shifted out to the login pageÐ
  if (!isAuthenticated) {Ð
    return <Navigate to="/login" replace />;Ð
  }Ð
Ð
  // Ø=Þ¨ 2. Authenticated non-admins get cleanly turned back away from internal administrative route spacesÐ
  if (requireAdmin && !isAdminUser) {Ð
    return <Navigate to="/" replace />;Ð
  }Ð
Ð
  // '  3. Security verification pass succeededÐ
  return children;Ð
}
```

File: src/components/ErrorBoundary.tsx

```
import { Component } from 'react';Ð
import type { ErrorInfo, ReactNode } from 'react';Ð
Ð
interface Props {Ð
  children: ReactNode;Ð
  fallback?: ReactNode;Ð
}Ð
Ð
interface State {Ð
  hasError: boolean;Ð
}Ð
Ð
export default class ErrorBoundary extends Component<Props, State> {Ð
  public state: State = {Ð
    hasError: falseÐ
  };Ð
Ð
  public static getDerivedStateFromError(_: Error): State {Ð
    // Ø=Þàþ  Update state so the next render will show the fallback UI.Ð
    return { hasError: true };Ð
  }Ð
Ð
  public componentDidCatch(error: Error, errorInfo: ErrorInfo) {Ð
    // Ø=Þ€ In production, you would stream this telemetry log to Sentry or LogRocketÐ
    console.error("Uncaught application rendering exception:", error, errorInfo);Ð
  }Ð
Ð
  private handleReset = () => {Ð
    this.setState({ hasError: false });Ð
    window.location.href = '/';Ð
  };Ð
Ð
  public render() {Ð
    if (this.state.hasError) {Ð
      if (this.props.fallback) {Ð
        return this.props.fallback;Ð
      }Ð
      return (Ð
        <div className="min-h-screen bg-slate-50 flex flex-col items-center justify-center p-4 font-sans">Ð
          <div className="max-w-md w-full bg-white border border-gray-200 rounded-2xl p-6 text-center shadow-
xs">Ð
            <div className="w-12 h-12 bg-red-50 text-red-600 rounded-full flex items-center justify-center
text-xl mx-auto mb-4 font-black">Ð
              &Ð
            </div>Ð
            <h2 className="text-base font-black text-gray-900 leading-tight mb-1">Ð
              Something went wrongÐ
            </h2>Ð
            <p className="text-xs text-gray-500 mb-6">Ð
              An unexpected layout processing error occurred. We have logged the issue and are looking into
it.Ð
            </p>Ð
            <buttonÐ
              onClick={this.handleReset}Ð
              className="w-full text-xs font-bold bg-gray-950 text-white hover:bg-gray-800 py-2.5 rounded-xl
transition shadow-2xs cursor-pointer"Ð
            >Ð
              Return to HomepageÐ
            </button>Ð
          </div>Ð
        </div>Ð
      );Ð
    }Ð
Ð
    return this.props.children;Ð
  }Ð
}
```

File: src/components/OrderSkeleton.tsx

```
  1 | Đ
  2 | Đ
  3 | export default function OrderSkeleton() {Đ
  4 |   return (Đ
  5 |     <div className="space-y-4 animate-pulse">Đ
  6 |       {[...Array(2)].map((_, idx) => (Đ
  7 |         <div key={idx} className="bg-white border border-gray-200 rounded-2xl shadow-sm overflow-hidden">Đ
  8 |           {/* Header Bar Skeleton */}Đ
  9 |           <div className="bg-gray-50 border-b border-gray-200 p-4 flex flex-wrap justify-between items-
center gap-2">Đ
 10 |             <div className="flex gap-4 w-2/3">Đ
 11 |               <div className="h-3 bg-gray-200 rounded-sm w-1/3"></div>Đ
 12 |               <div className="h-3 bg-gray-200 rounded-sm w-1/4"></div>Đ
 13 |               <div className="h-3 bg-gray-200 rounded-sm w-1/4"></div>Đ
 14 |             </div>Đ
 15 |             <div className="h-5 bg-gray-200 rounded-full w-20"></div>Đ
 16 |           </div>Đ
 17 | Đ
 18 |           {/* Item Row Skeleton */}Đ
 19 |           <div className="p-4 flex items-center justify-between gap-4">Đ
 20 |             <div className="flex items-center gap-3 w-3/4">Đ
 21 |               <div className="w-10 h-10 rounded-lg bg-gray-200 flex-shrink-0"></div>Đ
 22 |               <div className="space-y-2 w-1/2">Đ
 23 |                 <div className="h-4 bg-gray-200 rounded-sm w-full"></div>Đ
 24 |                 <div className="h-3 bg-gray-200 rounded-sm w-1/4"></div>Đ
 25 |               </div>Đ
 26 |             </div>Đ
 27 |             <div className="h-4 bg-gray-200 rounded-sm w-12"></div>Đ
 28 |           </div>Đ
 29 | Đ
 30 |           {/* Footer Bar Skeleton */}Đ
 31 |           <div className="bg-gray-50/50 border-t border-gray-150 px-4 py-2.5">Đ
 32 |             <div className="h-3 bg-gray-200 rounded-sm w-1/3"></div>Đ
 33 |           </div>Đ
 34 |         </div>Đ
 35 |       ))}Đ
 36 |     </div>Đ
 37 |   );Đ
 38 | }
```

File: src/components/ScrollToTop.tsx

```
import { useEffect } from 'react';Ð
import { useLocation } from 'react-router-dom';Ð
Ð
export default function ScrollToTop() {Ð
  //useLocation hooks into the router and detects whenever the URL path changesÐ
  const { pathname } = useLocation();Ð
Ð
  useEffect(() => {Ð
    // Instantly reset the window scroll coordinates to the top-left cornerÐ
    window.scrollTo(0, 0);Ð
  }, [pathname]); // This effect fires every single time a user switches pagesÐ
Ð
  return null; // This component doesn't render any visible UI elementÐ
}
```

File: src/pages/Home.tsx

```tsx
import { useState, useEffect } from 'react';
import { supabase } from '../supabaseClient';
import Navbar from '../components/Navbar';
import Hero from '../components/Hero';
import SearchBar from '../components/SearchBar';
import ProductGrid from '../components/ProductGrid';
import FeaturedLanes from '../components/FeaturedLanes';
import Subscribe from '../components/Subscribe';
import Footer from '../components/Footer';
import type { Product } from '../types/database';

export default function Home() {
  const [searchQuery, setSearchQuery] = useState<string>('');
  const [allProducts, setAllProducts] = useState<Product[]>([]);
  const [loading, setLoading] = useState<boolean>(false);
  const [selectedCategory, setSelectedCategory] = useState<string>('All');
  const [categoriesList, setCategoriesList] = useState<string[]>([]);

  const [currentPage, setCurrentPage] = useState<number>(1);
  const itemsPerPage = 6;

  const defaultSareeCategories = [
    'All',
    'Katan',
    'Rajshahi Silk',
    'Jamdani',
    'Georgette',
    'Muslin',
    'Organza',
    'Bridal Collection',
    'Chiffon',
  ];

  useEffect(() => {
    async function fetchDynamicCategories() {
      try {
        const { data, error } = await supabase
          .from('categories')
          .select('name')
          .order('name', { ascending: true });

        if (!error && data && data.length > 0) {
          setCategoriesList(['All', ...data.map((item) => item.name)]);
        } else {
          setCategoriesList(defaultSareeCategories);
        }
      } catch (err) {
        setCategoriesList(defaultSareeCategories);
      }
    }
    fetchDynamicCategories();
  }, []);

  useEffect(() => {
    setCurrentPage(1);
  }, [searchQuery, selectedCategory]);

  useEffect(() => {
    if (searchQuery.trim() !== '') {
      window.scrollTo({
        top: 0,
        behavior: 'smooth',
      });
    }
  }, [searchQuery]);

  useEffect(() => {
    const fetchProducts = async () => {
      setLoading(true);
      try {
        let queryBuilder = supabase.from('products').select('*');

        if (selectedCategory !== 'All') {
          queryBuilder = queryBuilder.ilike('category', selectedCategory);
        }

```

File: src/pages/Home.tsx (continued)

```
        const cleanedSearchQuery = searchQuery.trim();
        if (cleanedSearchQuery !== '') {
          queryBuilder = queryBuilder.ilike('title', `%${cleanedSearchQuery}%`);
        }

        queryBuilder = queryBuilder.order('id', { ascending: false });

        const { data, error } = await queryBuilder;
        if (!error && data) {
          setAllProducts(data as Product[]);
        }
      } catch (err) {
        console.error(err);
      } finally {
        setLoading(false);
      }
    };

    fetchProducts();
  }, [searchQuery, selectedCategory]);

  const isSearching = searchQuery.trim() !== '';
  const isFilteringCategory = selectedCategory !== 'All';
  const isSearchingOrFiltering = isSearching || isFilteringCategory;

  const startIndex = (currentPage - 1) * itemsPerPage;
  const paginatedProducts = allProducts.slice(startIndex, startIndex + itemsPerPage);
  const totalPages = Math.ceil(allProducts.length / itemsPerPage);

  return (
    <div className="min-h-screen flex flex-col bg-rose-50/20 font-sans">
      <div className="sticky top-0 z-50 bg-white/95 backdrop-blur-md shadow-xs border-b border-rose-100">
        <Navbar />
        <div className="max-w-6xl w-full mx-auto px-4 pb-2 pt-1">
          <SearchBar onSearch={setSearchQuery} isLoading={loading} />
        </div>
      </div>

      <main className="flex-1 max-w-6xl w-full mx-auto px-4 py-4 box-border">
        {isSearching && (
          <section id="search-view" className="my-6">
            <div className="flex items-center justify-between mb-6">
              <h2 className="text-lg sm:text-xl font-extrabold text-rose-950 tracking-tight">
                Results for "{searchQuery.trim()}"
              </h2>
              <span className="text-xs font-bold text-rose-800 bg-rose-100 px-3 py-1 rounded-full">
                {allProducts.length} Sarees Found
              </span>
            </div>

            <ProductGrid products={paginatedProducts} loading={loading} />
          </section>
        )}

        {!isSearching && <Hero />}

        {/* Category Pills Navigation */}
        <section className="my-8">
          <div className="flex flex-wrap justify-center gap-2 pb-4 border-b border-rose-100">
            {categoriesList.map((category) => (
              <button
                key={category}
                onClick={() => {
                  setSelectedCategory(category);
                  setSearchQuery('');
                }}
                className={`px-4 py-2 text-xs font-black text-black rounded-xl border uppercase tracking-wider transition-all duration-200 cursor-pointer ${
                  selectedCategory === category
                    ? 'bg-rose-900 text-white border-rose-900 shadow-xs'
                    : 'bg-white text-gray-700 border-rose-100 hover:border-rose-300 hover:bg-rose-50/50'
                }`}
              >
                {category}
              </button>
            ))}
          </div>
        </section>

```

File: src/pages/Home.tsx (continued)

```
        {!isSearching && (
          isFilteringCategory ? (
            <section id="category-view" className="my-6">
              <div className="flex items-center justify-between mb-6">
                <h2 className="text-lg sm:text-xl font-serif font-bold text-rose-950 tracking-tight uppercase">
                  {selectedCategory} Saree Collection
                </h2>
              </div>
              <ProductGrid products={paginatedProducts} loading={loading} />
            </section>
          ) : (
            <section className="my-6">
              {loading ? (
                <div className="py-20 text-center flex flex-col items-center justify-center gap-3">
                  <div className="w-8 h-8 border-4 border-rose-800 border-t-transparent rounded-full animate-spin"></div>
                  <p className="text-gray-400 text-xs font-bold uppercase tracking-wider">Loading Showcase...</p>
                </div>
              ) : (
                <FeaturedLanes
                  products={allProducts}
                  categories={categoriesList}
                  limitProducts={4}
                />
              )}
            </section>
          )
        )}

        {isSearchingOrFiltering && totalPages > 1 && !loading && (
          <div className="mt-10 mb-6 flex items-center justify-center gap-2 bg-white border border-rose-100 p-4 rounded-2xl shadow-xs">
            <button
              disabled={currentPage === 1}
              onClick={() => setCurrentPage((prev) => Math.max(1, prev - 1))}
              className="px-4 py-2 border border-gray-200 rounded-xl text-xs font-bold bg-rose-50 text-rose-950 hover:bg-rose-100 disabled:opacity-40 cursor-pointer"
            >
              !• Prev
            </button>
            <span className="text-xs font-black px-4 text-rose-950">
              Page {currentPage} of {totalPages}
            </span>
            <button
              disabled={currentPage === totalPages}
              onClick={() => setCurrentPage((prev) => Math.min(totalPages, prev + 1))}
              className="px-4 py-2 border border-gray-200 rounded-xl text-xs font-bold bg-rose-50 text-rose-950 hover:bg-rose-100 disabled:opacity-40 cursor-pointer"
            >
              Next !’
            </button>
          </div>
        )}

        <Subscribe />
      </main>

      <Footer />
    </div>
  );
}
```

File: src/pages/Products.tsx

```tsx
import { useState, useEffect } from 'react';
import { supabase } from '../supabaseClient';
import Navbar from '../components/Navbar';
import Footer from '../components/Footer';
import SearchBar from '../components/SearchBar';
import FeaturedLanes from '../components/FeaturedLanes';
import type { Product } from '../types/database';

export default function Products() {
  const [allProducts, setAllProducts] = useState<Product[]>([]);
  const [loading, setLoading] = useState<boolean>(true);
  const [searchQuery, setSearchQuery] = useState<string>('');
  const [selectedCategory, setSelectedCategory] = useState<string>('All');
  const [categories, setCategories] = useState<string[]>(['All']);

  const defaultCategories = [
    'All',
    'Katan',
    'Rajshahi Silk',
    'Jamdani',
    'Georgette',
    'Muslin',
    'Organza',
    'Bridal Collection',
    'Chiffon',
  ];

  useEffect(() => {
    async function fetchDynamicCategories() {
      try {
        const { data, error } = await supabase
          .from('categories')
          .select('name')
          .order('name', { ascending: true });

        if (!error && data && data.length > 0) {
          setCategories(['All', ...data.map((item) => item.name)]);
        } else {
          setCategories(defaultCategories);
        }
      } catch (err) {
        setCategories(defaultCategories);
      }
    }
    fetchDynamicCategories();
  }, []);

  useEffect(() => {
    const fetchInventory = async () => {
      setLoading(true);
      try {
        let query = supabase.from('products').select('*');

        if (selectedCategory !== 'All') {
          query = query.ilike('category', selectedCategory);
        }
        if (searchQuery.trim() !== '') {
          query = query.ilike('title', `%${searchQuery.trim()}%`);
        }

        query = query.order('id', { ascending: false });

        const { data, error } = await query;
        if (!error && data) {
          setAllProducts(data as Product[]);
        }
      } catch (err) {
        console.error(err);
      } finally {
        setLoading(false);
      }
    };

    fetchInventory();
  }, [searchQuery, selectedCategory]);

```

File: src/pages/Products.tsx (continued)

```
  const activeLanesList = selectedCategory === 'All' ? categories : [selectedCategory];

  return (
    <div className="min-h-screen flex flex-col bg-rose-50/20 font-sans">
      <Navbar />

      <main className="flex-1 max-w-6xl w-full mx-auto px-4 py-8">
        <div className="mb-8 text-center md:text-left">
          <h1 className="text-2xl sm:text-3xl font-serif font-bold text-rose-950 tracking-tight mb-2">
            Explore Saree Collections
          </h1>
          <p className="text-gray-500 text-xs sm:text-sm mb-6">
            Browse through our authentic repository of handcrafted saree weaves.
          </p>

          <div className="w-full max-w-2xl md:mx-0">
            <SearchBar onSearch={setSearchQuery} isLoading={loading} />
          </div>
        </div>

        {/* Filter Pills */}
        <div className="flex flex-wrap items-center justify-center md:justify-start gap-2 bg-white p-4 rounded-2xl border border-rose-100 shadow-xs mb-8">
          <span className="text-[11px] font-black uppercase tracking-wider text-rose-900 mr-2 hidden sm:inline">
            Weave Filter:
          </span>
          {categories.map((cat) => (
            <button
              key={cat}
              onClick={() => setSelectedCategory(cat)}
              className={`text-[11px] sm:text-xs font-bold px-4 py-2 rounded-xl border transition-all duration-200 cursor-pointer ${
                selectedCategory === cat
                  ? 'bg-rose-900 text-white border-rose-900 shadow-xs'
                  : 'bg-rose-50/50 text-gray-700 border-rose-100 hover:bg-rose-100/60'
              }`}
            >
              {cat}
            </button>
          ))}
        </div>

        <div className="w-full">
          {loading ? (
            <div className="py-20 text-center flex flex-col items-center justify-center gap-3">
              <div className="w-8 h-8 border-4 border-rose-900 border-t-transparent rounded-full animate-spin"></div>
              <p className="text-gray-400 text-xs font-bold uppercase tracking-wider">Syncing Saree Grid...</p>
            </div>
          ) : (
            <FeaturedLanes
              products={allProducts}
              categories={activeLanesList}
              limitProducts={4}
            />
          )}
        </div>
      </main>

      <Footer />
    </div>
  );
}
```

File: src/pages/ProductDetails.tsx

```tsx
import React, { useState, useEffect } from 'react';
import { Link, useParams, useNavigate } from 'react-router-dom';
import { supabase } from '../supabaseClient';
import Navbar from '../components/Navbar';
import Footer from '../components/Footer';
import { formatBDT } from '../types/database';
import type { Product, ProductComment } from '../types/database';
import { getErrorMessage } from '../utils/errorHandling';

export default function ProductDetails() {
  const { id } = useParams<{ id: string }>();
  const navigate = useNavigate();

  const [product, setProduct] = useState<Product | null>(null);
  const [loading, setLoading] = useState<boolean>(true);
  const [itemCount, setItemCount] = useState<number>(1);
  const [cartLoading, setCartLoading] = useState<boolean>(false);
  const [statusMessage, setStatusMessage] = useState<string>('');
  const [isAddedSuccess, setIsAddedSuccess] = useState<boolean>(false);

  // Dynamic Comments & Rating
  const [commentsList, setCommentsList] = useState<ProductComment[]>([]);
  const [averageRating, setAverageRating] = useState<number>(5.0);
  const [loadingComments, setLoadingComments] = useState<boolean>(false);

  // User input
  const [userRating, setUserRating] = useState<number>(5);
  const [userText, setUserText] = useState<string>('');
  const [submittingComment, setSubmittingComment] = useState<boolean>(false);

  const fetchCommentsAndRatings = async () => {
    setLoadingComments(true);
    try {
      const { data, error } = await supabase
        .from('product_comments')
        .select('id, product_id, user_id, user_email, rating, comment, created_at')
        .eq('product_id', Number(id))
        .order('created_at', { ascending: false });

      if (!error && data) {
        setCommentsList(data as ProductComment[]);
        if (data.length > 0) {
          const sum = data.reduce((acc, curr) => acc + curr.rating, 0);
          setAverageRating(parseFloat((sum / data.length).toFixed(1)));
        } else {
          setAverageRating(5.0);
        }
      }
    } catch (err) {
      console.error('Error reading reviews:', getErrorMessage(err));
    } finally {
      setLoadingComments(false);
    }
  };

  useEffect(() => {
    const fetchSingleItem = async () => {
      setLoading(true);
      setStatusMessage('');
      const { data, error } = await supabase
        .from('products')
        .select('*')
        .eq('id', Number(id))
        .single();

      if (error) {
        console.error('Error fetching product:', error.message);
      } else if (data) {
        setProduct(data as Product);
      }
      setLoading(false);
    };

    if (id) {
      fetchSingleItem();
      fetchCommentsAndRatings();
```

File: src/pages/ProductDetails.tsx (continued)

```
    }
  }, [id]);

  const handleAddToCart = async () => {
    setCartLoading(true);
    setStatusMessage('');

    const { data: { user } } = await supabase.auth.getUser();
    if (!user) {
      setStatusMessage('& þ  Please log in to add sarees to your cart.');
      setCartLoading(false);
      setTimeout(() => navigate('/login'), 1500);
      return;
    }

    try {
      // Check existing
      const { data: existing } = await supabase
        .from('cart_items')
        .select('id, quantity')
        .eq('user_id', user.id)
        .eq('product_id', Number(id))
        .maybeSingle();

      if (existing) {
        const { error } = await supabase
          .from('cart_items')
          .update({ quantity: existing.quantity + itemCount })
          .eq('id', existing.id);
        if (error) throw error;
      } else {
        const { error } = await supabase.from('cart_items').insert([
          {
            user_id: user.id,
            product_id: product?.id,
            quantity: itemCount,
          },
        ]);
        if (error) throw error;
      }

      setIsAddedSuccess(true);
      setTimeout(() => setIsAddedSuccess(false), 3000);
    } catch (err: unknown) {
      setStatusMessage(''L Error updating cart: ' + getErrorMessage(err));
    } finally {
      setCartLoading(false);
    }
  };

  const handleCommentSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    if (!userText.trim()) return;

    setSubmittingComment(true);
    try {
      const { data: { user } } = await supabase.auth.getUser();
      if (!user) {
        navigate('/login');
        return;
      }

      const chosenDisplayName =
        user.user_metadata?.display_name ||
        user.user_metadata?.full_name ||
        user.email?.split('@')[0] ||
        'Valued Customer';

      const commentPayload = {
        product_id: Number(id),
        user_id: user.id,
        user_email: chosenDisplayName,
        rating: userRating,
        comment: userText.trim(),
      };

      const { error } = await supabase
        .from('product_comments')
```

File: src/pages/ProductDetails.tsx (continued)

```
        .insert([commentPayload]);

      if (error) throw error;

      setUserText('');
      await fetchCommentsAndRatings();
    } catch (err: unknown) {
      alert(getErrorMessage(err) || 'Failed to submit review.');
    } finally {
      setSubmittingComment(false);
    }
  };

  if (loading) {
    return (
      <div className="min-h-screen flex flex-col justify-between bg-rose-50/20 font-sans">
        <Navbar />
        <div className="flex-1 flex flex-col items-center justify-center gap-2">
          <div className="w-8 h-8 border-4 border-rose-900 border-t-transparent rounded-full animate-spin"></
div>
          <p className="text-gray-400 text-xs font-bold uppercase tracking-wider">Loading Saree Details...</
p>
        </div>
        <Footer />
      </div>
    );
  }

  if (!product) {
    return (
      <div className="min-h-screen flex flex-col justify-between bg-rose-50/20 font-sans">
        <Navbar />
        <div className="text-center py-20 text-gray-500 font-medium">Saree record could not be found.</div>
        <Footer />
      </div>
    );
  }

  return (
    <div className="min-h-screen flex flex-col bg-rose-50/20 font-sans">
      <Navbar />

      <main className="flex-1 max-w-5xl w-full mx-auto px-3 sm:px-4 py-6 sm:py-12 space-y-6">

        {/* Navigation */}
        <div className="flex justify-start">
          <Link
            to="/products"
            className="inline-flex items-center gap-2 bg-rose-900 text-white hover:bg-rose-800 px-4 py-2
sm:px-5 sm:py-2.5>rounded-xl text-xs font-black uppercase tracking-wider transition active:scale-95 group"
            <span className="transition-transform group-hover:-translate-x-1">!•</span>
            <span>Back to Collections</span>
          </Link>
        </div>

        {statusMessage && !isAddedSuccess && (
          <div className="p-4 rounded-xl text-xs font-bold bg-white border border-rose-100 shadow-2xs">
            {statusMessage}
          </div>
        )}

        {/* Primary Details Matrix */}
        <div className="grid grid-cols-1 md:grid-cols-2 gap-6 sm:gap-10 bg-white border border-rose-100 p-4
sm:p-8 rounded-3xl shadow-xs">
          {/* Image Container */}
          <div className="w-full h-64 sm:h-96 bg-rose-50/30 rounded-2xl p-4 flex items-center justify-center
overflow-hidden border border-rose-100 shrink-0">
            <img src={product.image_url} alt={product.title} className="max-h-full max-w-full object-
contain" />
          </div>

          {/* Details & Actions */}
          <div className="flex flex-col justify-between min-w-0">
            <div className="space-y-4 sm:space-y-6">
              <div>
                <span className="inline-block bg-rose-100 text-rose-900 font-black text-[9px] sm:text-[10px]
tracking-widest uppercase px-3 py-1 rounded-md mb-2">
                  {product.category}
                </span>

                <h1 className="text-xl sm:text-2xl md:text-3xl font-serif font-bold text-rose-950 leading-
tight mb-2">
                  {product.title}
                </h1>
```

File: src/pages/ProductDetails.tsx (continued)

```
                {/* Rating */}
                <div className="flex items-center gap-1">
                  {[...Array(5)].map((_, i) => (
                    <span key={i} className={`text-sm sm:text-base ${i < Math.round(averageRating) ? 'text-amber-400' : 'text-gray-200'}`}>
                      &
                    </span>
                  ))}
                  <span className="text-xs text-gray-500 font-semibold ml-2">
                    ({averageRating} · {commentsList.length} customer reviews)
                  </span>
                </div>
              </div>

              {/* Price & Cart Actions */}
              <div className="bg-rose-50/40 border border-rose-100 p-4 sm:p-5 rounded-2xl flex flex-col gap-4">
                <div className="flex items-center justify-between border-b border-rose-100/80 pb-3">
                  <div className="flex flex-col">
                    <span className="text-[9px] sm:text-[10px] font-bold text-gray-400 uppercase tracking-widest">
                      Price
                    </span>
                    <span className="text-2xl sm:text-3xl font-black text-rose-950 font-mono">
                      {formatBDT(product.price)}
                    </span>
                  </div>

                  <div className="flex flex-col items-end">
                    <span className="text-[9px] sm:text-[10px] font-bold text-gray-400 uppercase tracking-widest">
                      Availability
                    </span>
                    <span className="text-xs font-black text-emerald-700 bg-emerald-100/70 border border-emerald-200 px-2.5 py-1 rounded-lg">
                      In Stock (Ready to Ship)
                    </span>
                  </div>
                </div>

                <div className="flex flex-wrap sm:flex-nowrap gap-3 items-center w-full">
                  {/* Stepper */}
                  <div className="flex items-center bg-white border border-gray-200 rounded-xl overflow-hidden shadow-2xs shrink-0 h-11">
                    <button
                      disabled={isAddedSuccess}
                      onClick={() => setItemCount(Math.max(1, itemCount - 1))}
                      className="w-10 h-full flex items-center justify-center font-black text-xl bg-rose-50 text-rose-700 hover:bg-rose-600 hover:text-white border-r border-gray-200 transition disabled:opacity-40 cursor-pointer"
                    >
                      -
                    </button>
                    <span className="px-3 text-sm font-black text-gray-900 min-w-[36px] text-center font-mono select-none">
                      {itemCount}
                    </span>
                    <button
                      disabled={isAddedSuccess}
                      onClick={() => setItemCount(itemCount + 1)}
                      className="w-10 h-full flex items-center justify-center font-black text-xl bg-emerald-50 text-emerald-700 hover:bg-emerald-600 hover:text-white border-l border-gray-200 transition disabled:opacity-40 cursor-pointer"
                    >
                      +
                    </button>
                  </div>

                  {/* Add Button */}
                  <div className="flex-1 flex flex-col relative min-w-0 h-11">
                    {isAddedSuccess && (
                      <span className="absolute -top-4 left-0 right-0 text-center text-[9px] font-bold text-emerald-600 animate-pulse">
                        Added to cart!
                      </span>
                    )}
                    <button
                      onClick={handleAddToCart}
                      disabled={cartLoading || isAddedSuccess}
                      className={`w-full h-full text-white font-black uppercase tracking-wider rounded-xl shadow-2xs transition cursor-pointer flex items-center justify-center text-xs px-4 ${
                        isAddedSuccess
                          ? 'bg-emerald-600 cursor-default'
                          : cartLoading
                          ? 'bg-rose-400 cursor-not-allowed'
                          : 'bg-rose-900 hover:bg-rose-800 active:scale-[0.98]'
                      }`}
                    >
                      {isAddedSuccess
                        ? ''  Added to Cart'
```

File: src/pages/ProductDetails.tsx (continued)

```
                        : cartLoading
                        ? 'Syncing...'
                        : `Add to Cart (${formatBDT(product.price * itemCount)})`}
                    </button>
                  </div>
                </div>
              </div>

              {/* Description */}
              <div className="border-t border-rose-50 pt-4">
                <h3 className="text-xs font-bold text-gray-400 uppercase tracking-wider mb-2">
                  Saree Craft & Specification
                </h3>
                <p className="text-xs sm:text-sm leading-relaxed text-gray-700">
                  {product.description}
                </p>
              </div>
            </div>
          </div>
        </div>

        {/* Reviews Section */}
        <div className="bg-white border border-rose-100 rounded-3xl p-4 sm:p-8 shadow-xs space-y-4">
          <h2 className="text-base sm:text-lg font-serif font-bold text-rose-950 uppercase tracking-tight
border-b border-rose-50 pb-2">
            Customer Reviews & Ratings
          </h2>

          <form onSubmit={handleCommentSubmit} className="bg-rose-50/40 border border-rose-100 p-4
rounded-2xl space-y-3">
            <h3 className="text-xs font-bold text-rose-950 uppercase tracking-wider">Write a Review</h3>

            <div className="flex items-center gap-2">
              <span className="text-xs font-medium text-gray-600">Your Rating:</span>
              <div className="flex gap-1">
                {[1, 2, 3, 4, 5].map((star) => (
                  <button
                    key={star}
                    type="button"
                    onClick={() => setUserRating(star)}
                    className={`text-base sm:text-xl select-none transition ${
                      star <= userRating ? 'text-amber-400 scale-105' : 'text-gray-200'
                    }`}
                  >
                    &
                  </button>
                ))}
              </div>
            </div>

            <div className="flex gap-2 items-center">
              <input
                type="text"
                value={userText}
                onChange={(e) => setUserText(e.target.value)}
                placeholder="Share your feedback regarding saree fabric quality, weave, or color..."
                className="flex-1 bg-white border border-gray-200 rounded-xl px-3 py-2 text-xs sm:text-sm
font-medium focus:outline-none focus:border-rose-500 shadow-2xs"
              />
              <button
                type="submit"
                disabled={submittingComment || !userText.trim()}
                className="bg-rose-900 text-white font-bold px-4 py-2 rounded-xl text-xs uppercase tracking-
wider hover:bg-rose-800 transition disabled:opacity-40 whitespace-nowrap shrink-0 cursor-pointer"
              >
                {submittingComment ? 'Posting...' : 'Submit'}
              </button>
            </div>
          </form>

          <div className="space-y-3 max-h-96 overflow-y-auto pr-1">
            {loadingComments ? (
              <p className="text-center text-xs text-gray-400 font-bold animate-pulse py-4">Loading customer
reviews...</p>
            ) : commentsList.length === 0 ? (
              <div className="text-center py-6 border border-dashed border-rose-100 rounded-2xl bg-
rose-50/20">
                <p className="text-xs text-gray-400 font-medium">No reviews submitted yet. Be the first to
review this saree!</p>
              </div>
            ) : (
              commentsList.map((record) => (
                <div
                  key={record.id}
                  className="bg-white border border-rose-100 p-3.5 rounded-2xl shadow-2xs flex flex-col
```

gap-1"

File: src/pages/ProductDetails.tsx (continued)

```
                >
                  <div className="flex justify-between items-center text-xs font-bold text-gray-400">
                    <span className="text-rose-950 truncate">{record.user_email}</span>
                    <span className="text-amber-400 text-xs">
                      {'& '.repeat(record.rating)}
                      {'& '.repeat(5 - record.rating)}
                    </span>
                  </div>
                  <p className="text-xs sm:text-sm font-medium text-gray-700 leading-normal break-words">
                    {record.comment}
                  </p>
                  <span className="text-[9px] text-gray-400 font-mono self-end">
                    {new Date(record.created_at).toLocaleDateString('en-BD', { dateStyle: 'medium' })}
                  </span>
                </div>
              ))
            )}
          </div>
        </div>

      </main>

      <Footer />
    </div>
  );
}
```

File: src/pages/Cart.tsx

```tsx
import { useEffect, useState } from 'react';
import { useNavigate } from 'react-router-dom';
import Navbar from '../components/Navbar';
import Footer from '../components/Footer';
import CheckoutForm from '../components/CheckoutForm';
import { supabase } from '../supabaseClient';
import { formatBDT } from '../types/database';
import { getErrorMessage } from '../utils/errorHandling';

interface CartItemRecord {
  id: number;
  quantity: number;
  products: {
    id: number;
    title: string;
    price: number;
    image_url: string;
  };
}

export default function Cart() {
  const navigate = useNavigate();

  const [cartItems, setCartItems] = useState<CartItemRecord[]>([]);
  const [loading, setLoading] = useState(true);
  const [uiError, setUiError] = useState<string | null>(null);
  const [uiSuccess, setUiSuccess] = useState<string | null>(null);

  // State controlling modal visibility
  const [isFormVisible, setIsFormVisible] = useState<boolean>(false);

  const fetchUserCart = async () => {
    setLoading(true);
    setUiError(null);
    try {
      const { data: { user } } = await supabase.auth.getUser();
      if (user) {
        const { data, error } = await supabase
          .from('cart_items')
          .select('id, quantity, products(id, title, price, image_url)')
          .eq('user_id', user.id);

        if (error) throw error;
        if (data) setCartItems(data as unknown as CartItemRecord[]);
      }
    } catch (err: unknown) {
      setUiError(getErrorMessage(err));
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => {
    fetchUserCart();
  }, []);

  const handleUpdateQuantity = async (itemId: number, newQty: number) => {
    if (newQty <= 0) {
      const { error } = await supabase.from('cart_items').delete().eq('id', itemId);
      if (!error) setCartItems((prev) => prev.filter((item) => item.id !== itemId));
    } else {
      const { error } = await supabase.from('cart_items').update({ quantity: newQty }).eq('id', itemId);
      if (!error) {
        setCartItems((prev) =>
          prev.map((item) => (item.id === itemId ? { ...item, quantity: newQty } : item))
        );
      }
    }
  };

  const handleRemoveItem = async (id: number) => {
    try {
      const { error } = await supabase.from('cart_items').delete().eq('id', id);
      if (error) throw error;
      setCartItems((prev) => prev.filter((item) => item.id !== id));
    } catch (err: unknown) {
```

File: src/pages/Cart.tsx (continued)

```
      setUiError('Could not remove item. Please try again: ' + getErrorMessage(err));
    }
  };

  const subtotal = cartItems.reduce((acc, item) => acc + item.products.price * item.quantity, 0);
  const shipping = subtotal > 0 ? 150 : 0; // Ÿ3 S  FVÆ—fW'’ 6† &vP
  const grandTotal = subtotal + shipping;

  const handleOrderSuccess = () => {
    setCartItems([]);
    setIsFormVisible(false);
    setUiSuccess('Ø<ß‰ Order placed successfully! Thank you for shopping with Menakkhi Sarees.');

    setTimeout(() => {
      navigate('/profile');
    }, 2000);
  };

  return (
    <div className="min-h-screen flex flex-col bg-rose-50/20 font-sans relative">
      <Navbar />

      <main className="flex-1 max-w-6xl w-full mx-auto px-4 py-8 sm:py-12">
        <h1 className="text-2xl sm:text-3xl font-black text-rose-950 tracking-tight mb-8">
          Shopping Cart ({cartItems.length} {cartItems.length === 1 ? 'Saree' : 'Sarees'})
        </h1>

        {uiError && !isFormVisible && (
          <div className="mb-6 p-4 bg-red-50 border border-red-200 text-red-600 rounded-2xl text-xs font-
bold">
            'L {uiError}
          </div>
        )}

        {uiSuccess && (
          <div className="mb-6 p-6 bg-emerald-50 border border-emerald-200 text-center rounded-2xl shadow-
xs">
            <span className="text-3xl block mb-2">Ø<ß‰</span>
            <h3 className="text-base font-black text-emerald-900 mb-1">{uiSuccess}</h3>
            <p className="text-xs text-emerald-700 font-medium">Redirecting to your orders dashboard...</p>
          </div>
        )}

        {loading ? (
          <div className="py-16 text-center border border-dashed border-rose-200 bg-white rounded-3xl">
            <div className="w-8 h-8 border-4 border-rose-800 border-t-transparent rounded-full animate-spin
mx-auto mb-3"></div>
            <p className="text-gray-500 font-medium text-xs">Loading your cart items...</p>
          </div>
        ) : cartItems.length === 0 && !uiSuccess ? (
          <div className="py-16 text-center border border-dashed border-rose-200 bg-white rounded-3xl p-6">
            <span className="text-5xl block mb-3">Ø=ÞÍþ </span>
            <p className="text-gray-500 font-medium text-sm mb-4">Your saree shopping cart is currently
empty</p>
            <button
              onClick={() => navigate('/products')}
              className="bg-rose-900 hover:bg-rose-800 text-white text-xs font-black uppercase tracking-
wider px-6 py-3 rounded-xl shadow-md transition cursor-pointer"
            >
              Explore Saree Collections
            </button>
          </div>
        ) : (
          cartItems.length > 0 && (
            <div className="grid grid-cols-1 lg:grid-cols-3 gap-8 items-start">
              {/* Left Column: Cart Items List */}
              <div className="lg:col-span-2 space-y-4">
                {cartItems.map((item) => (
                  <div
                    key={item.id}
                    className="bg-white border border-rose-100 rounded-2xl p-4 shadow-xs flex flex-col
sm:flex-row items-start sm:items-center justify-between gap-4"
                  >
                    {/* Image & Description */}
                    <div className="flex items-center gap-3.5 w-full sm:w-auto min-w-0">
                      <div className="h-20 w-20 rounded-xl bg-rose-50/50 flex-shrink-0 overflow-hidden flex
items-center justify-center border border-rose-100 p-1">
                        <img
                          src={item.products.image_url}
                          alt={item.products.title}
                          className="max-h-full max-w-full object-contain"
                        />
                      </div>
                      <div className="min-w-0 flex-1">
                        <h4 className="text-sm font-bold text-gray-900 truncate tracking-tight">
```

File: src/pages/Cart.tsx (continued)

```
                        {item.products.title}
                      </h4>
                      <p className="text-xs font-semibold text-rose-800 mt-0.5 font-mono">
                        {formatBDT(item.products.price)} each
                      </p>
                    </div>
                  </div>

                  {/* Quantity & Actions */}
                  <div className="flex flex-wrap items-center justify-between sm:justify-end gap-3 sm:gap-6 w-full sm:w-auto border-t sm:border-t-0 pt-3 sm:pt-0 border-rose-50">

                    {/* Quantity Control Stepper */}
                    <div className="flex items-center border border-gray-200 rounded-xl overflow-hidden bg-white shadow-2xs h-9">
                      <button
                        type="button"
                        onClick={() => handleUpdateQuantity(item.id, item.quantity - 1)}
                        className="w-8 h-full flex items-center justify-center text-base bg-rose-50 text-rose-700 hover:bg-rose-600 hover:text-white border-r border-gray-200 transition font-black cursor-pointer select-none"
                      >
                        -
                      </button>
                      <span className="px-3 text-xs font-black text-gray-900 min-w-[32px] text-center font-mono select-none">
                        {item.quantity}
                      </span>
                      <button
                        type="button"
                        onClick={() => handleUpdateQuantity(item.id, item.quantity + 1)}
                        className="w-8 h-full flex items-center justify-center text-base bg-emerald-50 text-emerald-700 hover:bg-emerald-600 hover:text-white border-l border-gray-200 transition font-black cursor-pointer select-none"
                      >
                        +
                      </button>
                    </div>

                    {/* Row Total */}
                    <div className="text-right sm:min-w-[90px]">
                      <span className="text-sm sm:text-base font-black text-rose-950 font-mono tracking-tight">
                        {formatBDT(item.products.price * item.quantity)}
                      </span>
                    </div>

                    {/* Remove Button */}
                    <button
                      type="button"
                      onClick={() => handleRemoveItem(item.id)}
                      className="p-1.5 rounded-lg text-gray-400 hover:text-red-600 hover:bg-red-50 transition cursor-pointer"
                      title="Remove Saree"
                    >
                      <svg className="h-4.5 w-4.5" fill="none" viewBox="0 0 24 24" stroke="currentColor" strokeWidth={2}>
                        <path strokeLinecap="round" strokeLinejoin="round" d="M19 7l-.867 12.142A2 2 0 0116.138 21H7.862a2 2 0 01-1.995-1.858L5 7m5 4v6m4-6v6m1-10V4a1 1 0 00-1-1h-4a1 1 0 00-1 1v3M4 7h16" />
                      </svg>
                    </button>
                  </div>
                </div>
              ))}
            </div>

            {/* Right Column: Summary Panel */}
            <div className="bg-white border border-rose-100 rounded-3xl p-6 shadow-sm">
              <h2 className="text-base font-black text-rose-950 tracking-tight mb-4">
                Order Summary
              </h2>

              <div className="space-y-3 text-xs sm:text-sm border-b border-rose-100 pb-4 mb-5">
                <div className="flex justify-between text-gray-600">
                  <span>Subtotal</span>
                  <span className="font-bold text-gray-900 font-mono">{formatBDT(subtotal)}</span>
                </div>
                <div className="flex justify-between text-gray-600">
                  <span>Delivery Charge (All BD)</span>
                  <span className="font-bold text-gray-900 font-mono">{formatBDT(shipping)}</span>
                </div>
                <div className="flex justify-between items-baseline pt-2 border-t border-gray-100">
                  <span className="text-sm font-black text-rose-950">Total Payable</span>
                  <span className="text-lg sm:text-xl font-black text-rose-700 font-mono">
                    {formatBDT(grandTotal)}
                  </span>
                </div>
              </div>

```

File: src/pages/Cart.tsx (continued)

```
                <button
                  type="button"
                  onClick={() => setIsFormVisible(true)}
                  className="w-full flex items-center justify-center gap-2 text-xs font-black uppercase tracking-wider bg-rose-900 text-white py-3.5 px-4 rounded-2xl shadow-md hover:bg-rose-800 transition cursor-pointer"
                >
                  <span>Proceed to Checkout</span>
                  <span>!’</span>
                </button>
              </div>
            </div>
          )
        )}
      </main>

      {/* CENTRIC OVERLAY MODAL */}
      {isFormVisible && (
        <div className="fixed inset-0 z-50 flex items-center justify-center p-4 bg-black/60 backdrop-blur-xs animate-in fade-in duration-200 overflow-y-auto">
          <div className="absolute inset-0 cursor-default" onClick={() => setIsFormVisible(false)}></div>
          <div className="relative z-10 w-full max-w-xl my-8">
            <CheckoutForm
              cartItems={cartItems}
              subtotal={subtotal}
              shipping={shipping}
              grandTotal={grandTotal}
              onClose={() => setIsFormVisible(false)}
              onOrderSuccess={handleOrderSuccess}
            />
          </div>
        </div>
      )}

      <Footer />
    </div>
  );
}
```

File: src/pages/CategoryPage.tsx

```
import { useEffect, useState } from 'react';
import { useParams, useNavigate } from 'react-router-dom';
import { supabase } from '../supabaseClient';
import Navbar from '../components/Navbar';
import Footer from '../components/Footer';
import ProductCard from '../components/ProductCard';
import type { Product } from '../types/database';
import { getErrorMessage } from '../utils/errorHandling';

export default function CategoryPage() {
  const { categoryName } = useParams<{ categoryName: string }>();
  const navigate = useNavigate();

  const [products, setProducts] = useState<Product[]>([]);
  const [loading, setLoading] = useState<boolean>(true);

  useEffect(() => {
    async function fetchCategoryProducts() {
      if (!categoryName) return;

      try {
        setLoading(true);
        const cleanCategoryString = decodeURIComponent(categoryName);

        const { data, error } = await supabase
          .from('products')
          .select('*')
          .ilike('category', cleanCategoryString);

        if (error) throw error;
        setProducts(data || []);
      } catch (err) {
        console.error('Error loading category items:', getErrorMessage(err));
      } finally {
        setLoading(false);
        window.scrollTo(0, 0);
      }
    }

    fetchCategoryProducts();
  }, [categoryName]);

  const displayTitle = categoryName
    ? decodeURIComponent(categoryName).charAt(0).toUpperCase() + decodeURIComponent(categoryName).slice(1)
    : '';

  return (
    <div className="min-h-screen flex flex-col bg-rose-50/20 font-sans">
      <Navbar />

      <main className="flex-1 max-w-7xl w-full mx-auto px-4 py-8 sm:py-12">
        <div className="mb-6 sm:mb-8 flex flex-col sm:flex-row sm:items-center sm:justify-between gap-4">
          <div>
            <button
              onClick={() => navigate('/')}
              className="text-xs font-bold text-rose-900 hover:text-rose-700 mb-2 block tracking-wide cursor-pointer"
            >
              !• Back to Main Showcase
            </button>
            <h1 className="text-2xl sm:text-3xl font-serif font-bold text-rose-950 tracking-tight uppercase">
              {displayTitle} Saree Collection
            </h1>
            <p className="text-gray-500 text-xs mt-1">
              Browsing handcrafted saree inventory filtered under the {displayTitle} taxonomy channel.
            </p>
          </div>
          <div className="bg-white border border-rose-100 text-rose-900 font-black text-xs px-4 py-2.5 rounded-xl shadow-2xs self-start sm:self-center">
            Ø<ß8 {products.length} Sarees Available
          </div>
        </div>

        {loading ? (
          <div className="flex justify-center items-center py-24">
            <div className="animate-spin rounded-full h-10 w-10 border-b-2 border-rose-900"></div>
          </div>
        ) : products.length === 0 ? (
```

File: src/pages/CategoryPage.tsx (continued)

```
  77 |           <div className="text-center py-20 bg-white rounded-3xl border border-rose-100 p-8 shadow-xs">
  78 |             <p className="text-gray-500 text-sm font-medium mb-4">
  79 |               No active saree items found inside the "{displayTitle}" collection.
  80 |             </p>
  81 |             <button
  82 |               onClick={() => navigate('/')}
  83 |               className="px-5 py-2.5 bg-rose-900 text-white text-xs font-black uppercase tracking-wider
rounded-xl hover:bg-rose-800 transition cursor-pointer"
  84 |
  85 |               Return Home
  86 |             </button>
  87 |           </div>
  88 |         ) : (
  89 |           <div className="grid grid-cols-2 md:grid-cols-3 lg:grid-cols-4 gap-3 sm:gap-6">
  90 |             {products.map((product) => (
  91 |               <ProductCard
  92 |                 key={product.id}
  93 |                 id={product.id}
  94 |                 title={product.title}
  95 |                 price={product.price}
  96 |                 image_url={product.image_url}
  97 |                 category={product.category}
  98 |                 description={product.description}
  99 |               />
 100 |             ))}
 101 |           </div>
 102 |         )}
 103 |       </main>
 104 |
 105 |       <Footer />
 106 |     </div>
 107 |   );
 108 | }
```

File: src/pages/Orders.tsx

```
import { useState, useEffect } from 'react';
import { useNavigate } from 'react-router-dom';
import { supabase } from '../supabaseClient';
import Navbar from '../components/Navbar';
import Footer from '../components/Footer';
import { formatBDT } from '../types/database';
import { getErrorMessage } from '../utils/errorHandling';

interface TopProductMetrics {
  id: number;
  product_name: string;
  image_url: string;
  price: number;
  unitsSold: number;
}

export default function Orders() {
  const navigate = useNavigate();
  const [topProducts, setTopProducts] = useState<TopProductMetrics[]>([]);
  const [loading, setLoading] = useState<boolean>(true);
  const [errorMessage, setErrorMessage] = useState<string | null>(null);

  useEffect(() => {
    const fetchTopSellingProducts = async () => {
      setLoading(true);
      setErrorMessage(null);
      try {
        const { data, error } = await supabase.rpc('get_public_top_sales');

        if (error) {
          console.error('Database Error:', error);
          setErrorMessage(`Database Error: ${error.message}`);
          return;
        }

        if (data) {
          const formattedProducts: TopProductMetrics[] = data.map((prod: any) => ({
            id: prod.id,
            product_name: prod.title,
            image_url: prod.image_url,
            price: prod.price,
            unitsSold: Number(prod.sales_count) || 0,
          }));

          setTopProducts(formattedProducts);
        }
      } catch (err: unknown) {
        setErrorMessage(getErrorMessage(err));
      } finally {
        setLoading(false);
      }
    };

    fetchTopSellingProducts();
  }, []);

  return (
    <div className="min-h-screen flex flex-col bg-rose-50/20 font-sans">
      <Navbar />

      <main className="flex-1 max-w-6xl w-full mx-auto px-4 py-8 sm:py-12">
        <div className="mb-8">
          <h1 className="text-2xl sm:text-3xl font-serif font-bold text-rose-950 tracking-tight mb-1">
            Top Sale Sarees
          </h1>
          <p className="text-gray-500 text-xs sm:text-sm">
            Explore the top trending and most loved saree weaves calculated across live purchase metrics.
          </p>
        </div>

        {errorMessage && (
          <div className="mb-6 p-4 bg-red-50 border border-red-200 text-red-600 rounded-2xl text-xs font-
bold">
            & þ  {errorMessage}
          </div>
        )}

```

File: src/pages/Orders.tsx (continued)

```
        {loading ? (
          <div className="py-24 text-center flex flex-col items-center justify-center gap-3 bg-white border border-rose-100 rounded-2xl shadow-2xs">
            <div className="w-8 h-8 border-4 border-rose-900 border-t-transparent rounded-full animate-spin"></div>
            <p className="text-gray-400 text-xs font-bold uppercase tracking-wider">Compiling trending saree catalog...</p>
          </div>
        ) : (
          <div>
            {topProducts.length === 0 ? (
              <div className="p-12 text-center bg-white border border-rose-100 text-gray-500 text-sm font-medium rounded-2xl">
                Ø=ÜÉ No saree purchase metrics logged yet. Check back later once orders are completed!
              </div>
            ) : (
              <div className="grid grid-cols-2 sm:grid-cols-3 md:grid-cols-4 lg:grid-cols-5 gap-3 sm:gap-6">
                {topProducts.map((prod, idx) => (
                  <div
                    key={prod.id}
                    onClick={() => navigate(`/product/${prod.id}`)}
                    className="bg-white border border-rose-100 rounded-2xl p-3 sm:p-4 flex flex-col relative shadow-2xs hover:shadow-lg transition duration-200 cursor-pointer group"
                  >
                    {/* Rank Badge */}
                    <span className="absolute top-3 left-3 z-10 bg-rose-950 text-amber-200 text-[9px] font-black px-2 py-0.5 rounded-md font-mono">
                      #{idx + 1}
                    </span>

                    {/* Sales Badge */}
                    <span className="absolute top-3 right-3 z-10 bg-rose-100 text-rose-900 text-[9px] font-black px-2 py-0.5 rounded-md font-mono">
                      {prod.unitsSold} Sold
                    </span>

                    {/* Image */}
                    <div className="h-36 sm:h-44 w-full rounded-xl bg-rose-50/40 flex items-center justify-center p-2 mb-3 overflow-hidden mt-5">
                      <img
                        src={prod.image_url}
                        alt={prod.product_name}
                        className="max-h-full max-w-full object-contain group-hover:scale-105 transition duration-300"
                      />
                    </div>

                    {/* Details */}
                    <div className="flex flex-col flex-1 min-w-0">
                      <h4
                        className="text-xs font-bold text-rose-950 line-clamp-2 leading-snug tracking-tight mb-2"
                        title={prod.product_name}
                      >
                        {prod.product_name}
                      </h4>
                      <div className="mt-auto pt-2 flex justify-between items-center border-t border-rose-50">
                        <span className="text-xs font-black text-rose-950 font-mono">
                          {formatBDT(prod.price)}
                        </span>
                        <span className="text-[10px] text-rose-700 font-bold opacity-0 group-hover:opacity-100 transition">
                          View !'
                        </span>
                      </div>
                    </div>
                  </div>
                ))}
              </div>
            )}
          </div>
        )}
      </main>

      <Footer />
    </div>
  );
}
```

File: src/pages/Login.tsx

```tsx
import React, { useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { supabase } from '../supabaseClient';
import Navbar from '../components/Navbar';
import Footer from '../components/Footer';
import { getErrorMessage } from '../utils/errorHandling';

export default function Login() {
  const navigate = useNavigate();
  const [email, setEmail] = useState<string>('');
  const [password, setPassword] = useState<string>('');
  const [name, setName] = useState<string>('');
  const [errorMsg, setErrorMsg] = useState<string | null>(null);
  const [loading, setLoading] = useState<boolean>(false);
  const [isSignUpMode, setIsSignUpMode] = useState<boolean>(false);

  const handleAuthSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setLoading(true);
    setErrorMsg(null);

    const targetEmail = email.trim().toLowerCase();

    try {
      if (isSignUpMode) {
        if (!name.trim()) {
          setErrorMsg('Please enter your full name.');
          setLoading(false);
          return;
        }

        // STEP 1: Register in Supabase Auth securely
        const { data, error: signUpError } = await supabase.auth.signUp({
          email: targetEmail,
          password: password,
          options: {
            data: {
              display_name: name.trim(),
            },
          },
        });

        if (signUpError) {
          if (signUpError.message.toLowerCase().includes('already registered')) {
            throw new Error('This email is already registered. Please switch to "Sign In".');
          }
          throw signUpError;
        }

        // STEP 2: Upsert into public.profiles (WITHOUT raw passwords!)
        if (data?.user) {
          await supabase.from('profiles').upsert([
            {
              id: data.user.id,
              name: name.trim(),
              email: targetEmail,
            },
          ]);
        }

        if (data?.session || data?.user) {
          navigate('/');
        } else {
          setErrorMsg('Account created successfully! Please sign in with your credentials.');
          setIsSignUpMode(false);
          setPassword('');
        }
      } else {
        // Sign In
        const { data, error: signInError } = await supabase.auth.signInWithPassword({
          email: targetEmail,
          password: password,
        });

        if (signInError) {
          if (signInError.message.toLowerCase().includes('invalid login credentials')) {
```

File: src/pages/Login.tsx (continued)

```
            throw new Error('Incorrect email or password. Please check your credentials or register a new account.');
          }
          throw signInError;
        }

        if (data?.user) {
          navigate('/');
        }
      }
    } catch (err: unknown) {
      console.error('Auth error:', getErrorMessage(err));
      setErrorMsg(getErrorMessage(err));
    } finally {
      setLoading(false);
    }
  };

  return (
    <div className="min-h-screen flex flex-col bg-rose-50/30 font-sans antialiased">
      <Navbar />

      <main className="flex-1 flex items-center justify-center p-6 relative overflow-hidden">
        {/* Background Gradients */}
        <div className="absolute top-1/4 left-1/3 w-72 h-72 bg-rose-300/20 rounded-full blur-3xl pointer-events-none"></div>
        <div className="absolute bottom-1/4 right-1/3 w-80 h-80 bg-amber-300/20 rounded-full blur-3xl pointer-events-none"></div>

        <div className="w-full max-w-md bg-white/90 backdrop-blur-xl border border-rose-100 rounded-3xl p-8 md:p-10 shadow-xl relative z-10">
          <div className="text-center mb-8">
            <div className="w-12 h-12 bg-rose-900 rounded-2xl flex items-center justify-center text-amber-300 text-xl font-bold mx-auto mb-4 shadow-md">
              &
            </div>
            <h1 className="text-2xl md:text-3xl font-serif font-bold text-rose-950 tracking-tight">
              {isSignUpMode ? 'Create Account' : 'Welcome Back'}
            </h1>
            <p className="text-xs md:text-sm text-gray-500 mt-1.5 font-medium">
              {isSignUpMode
                ? 'Join Menakkhi VIP Club for instant saree checkout'
                : 'Sign in to access your orders and saved saree items'}
            </p>
          </div>

          {errorMsg && (
            <div className="mb-6 bg-red-50 border border-red-200 rounded-xl p-3.5 flex items-start gap-2.5 text-xs font-semibold text-red-600">
              <span className="text-sm shrink-0">& þ </span>
              <p className="leading-relaxed">{errorMsg}</p>
            </div>
          )}

          <form onSubmit={handleAuthSubmit} className="space-y-4">
            {isSignUpMode && (
              <div>
                <label className="block text-[11px] font-bold text-gray-500 uppercase tracking-wider mb-1">
                  Full Name
                </label>
                <input
                  type="text"
                  required={isSignUpMode}
                  value={name}
                  onChange={(e) => setName(e.target.value)}
                  placeholder="e.g., Nusrat Jahan"
                  className="w-full text-xs md:text-sm border border-gray-200 bg-gray-50/50 px-4 py-3 rounded-xl focus:bg-white focus:outline-rose-500 font-medium"
                />
              </div>
            )}

            <div>
              <label className="block text-[11px] font-bold text-gray-500 uppercase tracking-wider mb-1">
                Email Address
              </label>
              <input
                type="email"
                required
                value={email}
                onChange={(e) => setEmail(e.target.value)}
                placeholder="name@example.com"
                className="w-full text-xs md:text-sm border border-gray-200 bg-gray-50/50 px-4 py-3 rounded-xl focus:bg-white focus:outline-rose-500 font-medium"
              />
```

File: src/pages/Login.tsx (continued)

```
            </div>

            <div>
              <label className="block text-[11px] font-bold text-gray-500 uppercase tracking-wider mb-1">
                Password
              </label>
              <input
                type="password"
                required
                value={password}
                onChange={(e) => setPassword(e.target.value)}
                placeholder="••••••••••••"
                className="w-full text-xs md:text-sm border border-gray-200 bg-gray-50/50 px-4 py-3 rounded-
xl focus:bg-white focus:outline-rose-500 font-mono tracking-widest"
              />
            </div>

            <div className="pt-2 space-y-4">
              <button
                type="submit"
                disabled={loading}
                className="w-full text-xs md:text-sm font-black uppercase tracking-wider bg-rose-900
hover:bg-rose-800 disabled:bg-rose-400 text-white py-3.5 rounded-2xl shadow-md transition active:scale-[0.98]
cursor-pointer text-center"
              >
                {loading ? 'Processing...' : isSignUpMode ? 'Register Account' : 'Sign In'}
              </button>

              <div className="text-center pt-2 border-t border-gray-100">
                <p className="text-xs text-gray-500 font-medium">
                  {isSignUpMode ? 'Already registered? ' : "Don't have an account yet? "}
                  <button
                    type="button"
                    onClick={() => {
                      setIsSignUpMode(!isSignUpMode);
                      setErrorMsg(null);
                    }}
                    className="text-rose-900 font-bold underline transition cursor-pointer"
                  >
                    {isSignUpMode ? 'Sign In' : 'Register'}
                  </button>
                </p>
              </div>
            </div>
          </form>
        </div>
      </main>

      <Footer />
    </div>
  );
}
```

File: src/pages/Register.tsx

```tsx
import React, { useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { supabase } from '../supabaseClient';
import Navbar from '../components/Navbar';
import Footer from '../components/Footer';
import { getErrorMessage } from '../utils/errorHandling';

export default function Register() {
  const navigate = useNavigate();
  const [fullName, setFullName] = useState<string>('');
  const [email, setEmail] = useState<string>('');
  const [password, setPassword] = useState<string>('');
  const [errorMsg, setErrorMsg] = useState<string | null>(null);
  const [successMsg, setSuccessMsg] = useState<string | null>(null);
  const [loading, setLoading] = useState<boolean>(false);

  const handleRegister = async (e: React.FormEvent) => {
    e.preventDefault();
    setLoading(true);
    setErrorMsg(null);
    setSuccessMsg(null);

    const targetEmail = email.trim().toLowerCase();

    try {
      // STEP 1: Supabase Authentication
      const { data: authData, error: authError } = await supabase.auth.signUp({
        email: targetEmail,
        password: password,
        options: {
          data: {
            display_name: fullName.trim(),
          },
        },
      });

      if (authError) {
        if (authError.message.toLowerCase().includes('already registered')) {
          throw new Error('This email address is already registered in our system.');
        }
        throw authError;
      }

      // STEP 2: Upsert metadata into public.profiles (WITHOUT raw password!)
      if (authData?.user) {
        const { error: dbError } = await supabase
          .from('profiles')
          .upsert([
            {
              id: authData.user.id,
              name: fullName.trim(),
              email: targetEmail,
            },
          ]);

        if (dbError) {
          console.error('Profile creation drop:', dbError.message);
        }
      }

      setSuccessMsg('Account registered successfully! Redirecting to home...');
      setFullName('');
      setEmail('');
      setPassword('');

      setTimeout(() => {
        navigate('/');
      }, 2000);
    } catch (err: unknown) {
      console.error(err);
      setErrorMsg(getErrorMessage(err) || 'Registration failed. Please try again.');
    } finally {
      setLoading(false);
    }
  };

```

File: src/pages/Register.tsx (continued)

```
  return (
    <div className="min-h-screen flex flex-col bg-rose-50/30 font-sans antialiased">
      <Navbar />

      <main className="flex-1 flex items-center justify-center p-6 relative overflow-hidden">
        <div className="w-full max-w-md bg-white/90 backdrop-blur-xl border border-rose-100 rounded-3xl p-8 md:p-10 shadow-xl relative z-10">

          <div className="text-center mb-8">
            <div className="w-12 h-12 bg-rose-900 rounded-2xl flex items-center justify-center text-amber-300 text-xl font-bold mx-auto mb-4 shadow-md">
              ð
            </div>
            <h1 className="text-2xl md:text-3xl font-serif font-bold text-rose-950 tracking-tight">
              Create Account
            </h1>
            <p className="text-xs md:text-sm text-gray-500 mt-1.5 font-medium">
              Join Menakkhi Sarees for seamless checkout and order updates
            </p>
          </div>

          {errorMsg && (
            <div className="mb-6 bg-red-50 border border-red-200 rounded-xl p-3.5 flex items-start gap-2.5 text-xs font-semibold text-red-600">
              <span className="text-sm shrink-0">& þ </span>
              <p className="leading-relaxed">{errorMsg}</p>
            </div>
          )}

          {successMsg && (
            <div className="mb-6 bg-emerald-50 border border-emerald-200 rounded-xl p-3.5 flex items-start gap-2.5 text-xs font-semibold text-emerald-700">
              <span className="text-sm shrink-0">Ø<Ûª</span>
              <p className="leading-relaxed">{successMsg}</p>
            </div>
          )}

          <form onSubmit={handleRegister} className="space-y-4">
            <div>
              <label className="block text-[11px] font-bold text-gray-500 uppercase tracking-wider mb-1">
                Full Name
              </label>
              <input
                type="text"
                required
                value={fullName}
                onChange={(e) => setFullName(e.target.value)}
                placeholder="e.g., Nusrat Jahan"
                className="w-full text-xs md:text-sm border border-gray-200 bg-gray-50/50 px-4 py-3 rounded-xl focus:bg-white focus:outline-rose-500 font-medium"
              />
            </div>

            <div>
              <label className="block text-[11px] font-bold text-gray-500 uppercase tracking-wider mb-1">
                Email Address
              </label>
              <input
                type="email"
                required
                value={email}
                onChange={(e) => setEmail(e.target.value)}
                placeholder="name@example.com"
                className="w-full text-xs md:text-sm border border-gray-200 bg-gray-50/50 px-4 py-3 rounded-xl focus:bg-white focus:outline-rose-500 font-medium"
              />
            </div>

            <div>
              <label className="block text-[11px] font-bold text-gray-500 uppercase tracking-wider mb-1">
                Secure Password
              </label>
              <input
                type="password"
                required
                value={password}
                onChange={(e) => setPassword(e.target.value)}
                placeholder="••••••••••••"
                className="w-full text-xs md:text-sm border border-gray-200 bg-gray-50/50 px-4 py-3 rounded-xl focus:bg-white focus:outline-rose-500 font-mono tracking-widest"
              />
            </div>

            <div className="pt-2 space-y-4">
              <button
```

File: src/pages/Register.tsx (continued)

```
                type="submit"
                disabled={loading}
                className="w-full text-xs md:text-sm font-black uppercase tracking-wider bg-rose-900
hover:bg-rose-800 disabled:bg-rose-400 text-white py-3.5 rounded-2xl shadow-md transition active:scale-[0.98]
cursor-pointer text-center"
              >
                {loading ? 'Creating Account...' : 'Register Account'}
              </button>

              <div className="text-center pt-2 border-t border-gray-100">
                <p className="text-xs text-gray-500 font-medium">
                  Already have an account?{' '}
                  <button
                    type="button"
                    onClick={() => navigate('/login')}
                    className="text-rose-900 font-bold underline transition cursor-pointer"
                  >
                    Sign In
                  </button>
                </p>
              </div>
            </div>
          </form>
        </div>
      </main>

      <Footer />
    </div>
  );
}
```

File: src/pages/Profile.tsx

```tsx
import { useEffect, useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { supabase } from '../supabaseClient';
import Navbar from '../components/Navbar';
import Footer from '../components/Footer';
import OrderSkeleton from '../components/OrderSkeleton';
import { useNetworkStatus } from '../hooks/useNetworkStatus';
import { fetchWithRetry } from '../utils/networkRetry';
import { formatBDT } from '../types/database';
import type { UserOrder } from '../types/database';
import { getErrorMessage } from '../utils/errorHandling';

export default function Profile() {
  const navigate = useNavigate();
  const isOnline = useNetworkStatus();
  const [orders, setOrders] = useState<UserOrder[]>([]);
  const [userEmail, setUserEmail] = useState<string | null>(null);
  const [loading, setLoading] = useState<boolean>(true);
  const [networkError, setNetworkError] = useState<string | null>(null);

  const [activeInvoiceOrder, setActiveInvoiceOrder] = useState<any | null>(null);

  useEffect(() => {
    let isMounted = true;

    const fetchUserDataAndOrders = async () => {
      if (!isOnline) {
        setLoading(false);
        return;
      }

      try {
        if (!isMounted) return;
        setLoading(true);
        setNetworkError(null);

        const { data: { user }, error: authError } = await supabase.auth.getUser();
        if (authError) throw authError;

        if (user) {
          if (isMounted) setUserEmail(user.email ?? 'Valued Customer');

          const data = await fetchWithRetry(async () => {
            const { data: res, error } = await supabase
              .from('orders')
              .select(`
                id,
                total_amount,
                status,
                shipping_address,
                created_at,
                payment_details,
                shipping_destination,
                contact_number,
                order_items!inner (
                  quantity,
                  price_at_purchase,
                  products!inner ( id, title, image_url )
                )
              `)
              .eq('user_id', user.id)
              .order('created_at', { ascending: false });

            if (error) throw error;
            return res;
          });

          if (isMounted && data) {
            setOrders(data as unknown as UserOrder[]);
          }
        }
      } catch (err: unknown) {
        if (isMounted) {
          const cleanMessage = getErrorMessage(err);
          setNetworkError('Unable to fetch order history: ' + cleanMessage);
        }
```

File: src/pages/Profile.tsx (continued)

```
      } finally {
        if (isMounted) setLoading(false);
      }
    };

    fetchUserDataAndOrders();

    return () => {
      isMounted = false;
    };
  }, [isOnline]);

  const handleLogout = async () => {
    try {
      await supabase.auth.signOut();
      window.location.href = '/';
    } catch (err: unknown) {
      console.error('Signout error:', getErrorMessage(err));
    }
  };

  const triggerSystemPrint = () => {
    window.print();
  };

  return (
    <div className="min-h-screen flex flex-col bg-rose-50/20 font-sans relative">
      <style>{`
        @media print {
          body * {
            visibility: hidden !important;
          }
          html, body {
            height: 99vh !important;
            overflow: hidden !important;
            background: #fff !important;
            font-size: 14px !important;
          }
          #drive-invoice-viewport, #drive-invoice-viewport * {
            visibility: visible !important;
          }
          #drive-invoice-viewport {
            position: absolute !important;
            left: 0 !important;
            top: 0 !important;
            width: 100% !important;
            max-width: 100% !important;
            box-shadow: none !important;
            border: none !important;
            padding: 0 !important;
            margin: 0 !important;
          }
          .no-print-action {
            display: none !important;
          }
        }
      `}</style>

      <div className="flex flex-col min-h-screen w-full print:hidden">
        <Navbar />

        <main className="flex-1 max-w-4xl w-full mx-auto px-4 py-8 sm:py-12">
          {!isOnline && (
            <div className="mb-6 p-3 bg-amber-500 text-white rounded-xl text-xs font-black tracking-wide uppercase text-center animate-pulse">
              &gt;_ Operating In Offline Mode.
            </div>
          )}

          {networkError && isOnline && (
            <div className="mb-6 p-3 bg-red-50 text-red-700 border border-red-200 rounded-xl text-xs font-bold text-center">
              {networkError}
            </div>
          )}

          {/* Account Profile Card */}
          <div className="bg-white border border-rose-100 p-6 rounded-3xl shadow-xs mb-8 flex flex-col sm:flex-row sm:items-center justify-between gap-4">
            <div className="flex items-center gap-4">
              <div className="w-12 h-12 rounded-2xl bg-rose-100 text-rose-950 flex items-center justify-
```

center font-black text-xl">

File: src/pages/Profile.tsx (continued)

```
                Ø<ß8
              </div>
              <div>
                <h2 className="text-base font-black text-rose-950 leading-tight">Customer Account</h2>
                <p className="text-xs text-gray-500 mt-0.5">{userEmail || 'Not Signed In'}</p>
              </div>
            </div>

            {userEmail ? (
              <button
                disabled={!isOnline}
                onClick={handleLogout}
                className={`text-xs font-black uppercase tracking-wider px-4 py-2.5 rounded-xl transition
cursor-pointer self-start sm:self-center ${
                  isOnline
                    ? 'text-red-700 bg-red-50 hover:bg-red-100'
                    : 'bg-gray-100 text-gray-400 cursor-not-allowed'
                }`}
              >
                Sign Out
              </button>
            ) : (
              <button
                onClick={() => navigate('/login')}
                className="text-xs font-black uppercase tracking-wider text-rose-900 bg-rose-50 hover:bg-
rose-100 px-4 py-2.5 rounded-xl transition cursor-pointer self-start sm:self-center"
              >
                Sign In
              </button>
            )}
          </div>

          {/* Order History */}
          <h3 className="text-sm font-black text-rose-950 tracking-wider uppercase mb-4">
            Your Saree Order Dispatches ({orders.length})
          </h3>

          {loading ? (
            <OrderSkeleton />
          ) : !userEmail ? (
            <div className="text-center py-16 bg-white border border-rose-100 rounded-3xl text-sm text-
gray-400">
              Please log in to view your saree order history.
            </div>
          ) : orders.length === 0 ? (
            <div className="text-center py-16 bg-white border border-rose-100 rounded-3xl text-sm text-
gray-400">
              You haven't placed any saree orders yet.
            </div>
          ) : (
            <div className="space-y-4">
              {orders.map((order) => (
                <div key={order.id} className="bg-white border border-rose-100 rounded-2xl shadow-2xs
overflow-hidden">
                  <div className="bg-rose-50/40 border-b border-rose-100 p-4 flex flex-wrap justify-between
items-center gap-2 text-xs">
                    <div className="flex flex-wrap gap-4 text-gray-600 font-medium">
                      <div>
                        <span>ORDER DATE: </span>
                        <span className="text-rose-950 font-bold">{new
Date(order.created_at).toLocaleDateString('en-BD')}</span>
                      </div>
                      <div>
                        <span>TOTAL: </span>
                        <span className="text-rose-950 font-bold font-mono">{formatBDT(order.total_amount)}</
span>
                      </div>
                      <div>
                        <span>ORDER REF: </span>
                        <span className="text-rose-950 font-bold font-mono">#{order.id}</span>
                      </div>
                    </div>
                    <div className="flex items-center gap-2">
                      <button
                        onClick={() => {
                          setActiveInvoiceOrder(order);
                          document.body.style.overflow = 'hidden';
                        }}
                        className="px-3 py-1.5 text-[11px] font-black rounded-xl bg-white text-rose-900
border border-rose-200 hover:bg-rose-100 transition cursor-pointer"
                      >
                        Ø=ÜÄ View Voucher
                      </button>
                      <span
                        className={`px-3 py-1 font-black text-[10px] uppercase rounded-full border ${
                          order.status === 'Shipped' || order.status === 'Delivered'
                            ? 'bg-emerald-50 text-emerald-800 border-emerald-200'
```

File: src/pages/Profile.tsx (continued)

```
                            : 'bg-amber-50 text-amber-800 border-amber-200'
                        }`}
                      >
                        {order.status === 'Shipped' || order.status === 'Delivered' ? 'Ø=Üæ Delivered' : '#ó Pending'}
                      </span>
                    </div>
                  </div>

                  <div className="divide-y divide-rose-50">
                    {order.order_items.map((item, index) => (
                      <div key={index} className="p-4 flex items-center justify-between gap-4 text-xs sm:text-sm">
                        <div className="flex items-center gap-3">
                          {item.products?.image_url ? (
                            <img
                              src={item.products.image_url}
                              alt={item.products.title}
                              className="w-12 h-12 rounded-xl object-contain bg-rose-50/40 border border-rose-100 flex-shrink-0"
                            />
                          ) : (
                            <div className="w-12 h-12 rounded-xl bg-gray-100 flex items-center justify-center text-xs text-gray-400">Ø=Üæ</div>
                          )}
                          <div>
                            <h4 className="font-bold text-rose-950 leading-tight">{item.products?.title || 'Catalog Saree'}</h4>
                            <p className="text-xs text-gray-400 mt-0.5">Quantity: {item.quantity}</p>
                          </div>
                        </div>
                        <span className="font-black text-rose-950 font-mono">
                          {formatBDT(item.price_at_purchase * item.quantity)}
                        </span>
                      </div>
                    ))}
                  </div>

                  <div className="bg-gray-50/50 border-t border-gray-100 px-4 py-2.5 text-[11px] text-gray-600 font-medium">
                    Ø=ÜÍ Delivery Address: <span className="text-gray-900 font-bold">{order.shipping_address}</span>
                  </div>
                </div>
              ))}
            </div>
          )}
        </main>

        <Footer />
      </div>

      {/* PRINTABLE VOUCHER MODAL */}
      {activeInvoiceOrder && (
        <div className="fixed inset-0 z-50 bg-white flex flex-col justify-start items-center overflow-y-auto p-2 sm:p-6 select-none animate-in fade-in duration-200">
          <div className="no-print-action max-w-4xl w-[95vw] bg-rose-950 text-white rounded-t-2xl px-4 py-3 border-b border-rose-800 flex justify-between items-center gap-2 shadow-xl">
            <div className="flex items-center gap-2.5 truncate">
              <span className="text-xl shrink-0">Ø=ÜÄ</span>
              <div className="truncate text-left">
                <p className="text-xs sm:text-sm font-bold truncate">Menakkhi_Voucher_#000{activeInvoiceOrder.id}.pdf</p>
                <p className="text-[10px] text-rose-200/70 font-semibold">Official Customer Purchase Receipt</p>
              </div>
            </div>
            <div className="flex items-center gap-2 shrink-0">
              <button
                onClick={triggerSystemPrint}
                className="px-4 py-1.5 bg-amber-400 hover:bg-amber-300 text-rose-950 font-black text-xs uppercase tracking-wider rounded-xl transition cursor-pointer shadow-md"
              >
                Ø=Üå Print Voucher
              </button>
              <button
                onClick={() => {
                  setActiveInvoiceOrder(null);
                  document.body.style.overflow = 'unset';
                }}
                className="p-1.5 bg-rose-900 hover:bg-rose-800 rounded-xl transition cursor-pointer text-sm text-rose-200"
                title="Close"
              >
                '
              </button>
            </div>
          </div>

          <div
            id="drive-invoice-viewport"
```

File: src/pages/Profile.tsx (continued)

```
            className="max-w-4xl w-[95vw] bg-white text-black p-4 sm:p-12 shadow-sm rounded-b-2xl min-h-[70vh] flex flex-col justify-between font-sans leading-relaxed text-sm tracking-normal border border-rose-100 border-t-0"
          >
            <div>
              <div className="flex flex-col sm:flex-row justify-between items-start border-b-4 border-rose-950 pb-4 mb-6 gap-4">
                <div>
                  <h1 className="text-2xl sm:text-4xl font-serif font-bold uppercase tracking-tight text-rose-950">MENAKKHI SAREES</h1>
                  <p className="text-xs font-bold text-rose-800 mt-0.5">Traditional & Modern Heritage Saree Boutique</p>
                </div>
                <div className="text-left sm:text-right">
                  <h2 className="text-xl sm:text-2xl font-black tracking-tight text-gray-900">PURCHASE VOUCHER</h2>
                  <p className="text-sm sm:text-base font-black text-rose-900 mt-0.5">Sequence: #000{activeInvoiceOrder.id}</p>
                  <p className="text-xs text-gray-400 font-bold mt-1">
                    Date: {new Date(activeInvoiceOrder.created_at).toLocaleDateString('en-BD', { year: 'numeric', month: 'long', day: 'numeric' })}
                  </p>
                </div>
              </div>

              <div className="grid grid-cols-1 sm:grid-cols-2 gap-4 mb-6 bg-rose-50/40 border border-rose-100 rounded-2xl p-4">
                <div>
                  <h3 className="text-[10px] font-black text-rose-900 uppercase tracking-widest mb-1">Customer Account</h3>
                  <p className="text-sm font-black text-gray-900">Verified Member</p>
                  <p className="text-xs text-gray-600 mt-0.5">' þ  {userEmail || 'N/A'}</p>
                  {activeInvoiceOrder.contact_number && (
                    <p className="text-xs font-mono font-bold text-rose-900 mt-1">Ø=ÜÞ {activeInvoiceOrder.contact_number}</p>
                  )}
                </div>
                <div>
                  <h3 className="text-[10px] font-black text-rose-900 uppercase tracking-widest mb-1">Shipping Location</h3>
                  <p className="text-xs text-gray-800 font-medium leading-relaxed">
                    {activeInvoiceOrder.shipping_address}
                  </p>
                </div>
              </div>

              <div className="mb-6">
                <h3 className="text-[10px] font-black text-rose-900 uppercase tracking-widest mb-2.5">Saree Line Items</h3>
                <table className="w-full text-left text-xs sm:text-sm">
                  <thead>
                    <tr className="border-b-2 border-rose-950 font-black text-rose-950 bg-rose-50">
                      <th className="py-2.5 px-3 w-[50%]">Saree Title</th>
                      <th className="py-2.5 px-3 text-center w-[15%]">Qty</th>
                      <th className="py-2.5 px-3 text-right w-[35%]">Price</th>
                    </tr>
                  </thead>
                  <tbody className="divide-y divide-rose-50">
                    {activeInvoiceOrder.order_items.map((item: any, index: number) => (
                      <tr key={index} className="font-semibold text-gray-800">
                        <td className="py-3 px-3 font-bold text-rose-950">{item.products?.title || 'Saree Item'}</td>
                        <td className="py-3 px-3 text-center font-mono">{item.quantity}</td>
                        <td className="py-3 px-3 text-right font-mono font-black text-rose-950">
                          {formatBDT(item.quantity * item.price_at_purchase)}
                        </td>
                      </tr>
                    ))}
                  </tbody>
                </table>
              </div>
            </div>

            <div className="mt-auto pt-6 border-t-2 border-rose-950">
              <div className="flex flex-col sm:flex-row justify-between items-start sm:items-end gap-4">
                <div>
                  <h3 className="text-[10px] font-black text-rose-900 uppercase tracking-widest mb-1">Payment Method</h3>
                  <span className="text-xs font-black bg-rose-100 text-rose-950 border border-rose-200 px-3 py-1 rounded-xl uppercase">
                    {activeInvoiceOrder.payment_details?.method === 'COD'
                      ? 'Ø=Üµ Cash On Delivery'
                      : activeInvoiceOrder.payment_details?.method === 'bKash'
                      ? 'Ø<ß8 bKash Personal'
                      : 'Ø=Ý% Nagad Personal'}
                  </span>
                  {activeInvoiceOrder.payment_details?.trx_id && activeInvoiceOrder.payment_details.method !== 'COD' && (
                    <p className="text-xs font-mono font-bold text-gray-600 mt-2">
                      TrxID: {activeInvoiceOrder.payment_details.trx_id}
                    </p>
                  )}
                </div>

                <div className="text-right w-full sm:w-auto min-w-[200px] space-y-1">
```

File: src/pages/Profile.tsx (continued)

```
                    <div className="flex justify-between text-xs font-bold text-gray-600">
                      <span>Subtotal:</span>
                      <span className="font-mono">{formatBDT(activeInvoiceOrder.total_amount)}</span>
                    </div>
                    <div className="flex justify-between items-end pt-2 border-t border-gray-200">
                      <span className="text-xs font-black text-rose-950 uppercase tracking-wider">Grand Total</span>
                      <span className="text-xl font-black text-rose-950 font-mono">
                        {formatBDT(activeInvoiceOrder.total_amount)}
                      </span>
                    </div>
                  </div>
                </div>

                <div className="mt-8 text-center text-[9px] text-gray-400 font-bold border-t border-rose-50 pt-3 uppercase tracking-widest">
                  Menakhi Sarees Official Customer Voucher • Thank you for your purchase
                </div>
              </div>
            </div>
          </div>
        )}
      </div>
    );
}
```

File: src/pages/Admin.tsx

```tsx
import { useEffect, useState } from 'react';
import { supabase } from '../supabaseClient';
import Navbar from '../components/Navbar';
import Footer from '../components/Footer';
import { formatBDT } from '../types/database';
import type { Product } from '../types/database';
import { getErrorMessage } from '../utils/errorHandling';

interface OrderItem {
  quantity: number;
  price_at_purchase: number;
  products: {
    id: number;
    title: string;
  } | null;
}

interface AdminOrder {
  id: number;
  total_amount: number;
  status: string;
  shipping_address: string;
  created_at: string;
  contact_number: string | null;
  payment_details: { method: string; trx_id: string } | null;
  shipping_destination: {
    type: 'Standard' | 'University';
    district?: string;
    upazila?: string;
    villageArea?: string;
    universityName?: string;
    hallName?: string;
  } | null;
  order_items: OrderItem[];
  customer_name?: string;
  customer_email?: string;
}

interface Category {
  id: string;
  name: string;
}

export default function Admin() {
  const [isAdminAuthenticated, setIsAdminAuthenticated] = useState<boolean>(false);
  const [adminPassword, setAdminPassword] = useState<string>('');
  const [authError, setAuthError] = useState<string>('');
  const [activeTab, setActiveTab] = useState<'orders' | 'inventory'>('orders');

  const [orders, setOrders] = useState<AdminOrder[]>([]);
  const [products, setProducts] = useState<Product[]>([]);
  const [categories, setCategories] = useState<Category[]>([]);
  const [loading, setLoading] = useState<boolean>(true);
  const [formLoading, setFormLoading] = useState<boolean>(false);

  const [activeInvoiceOrder, setActiveInvoiceOrder] = useState<AdminOrder | null>(null);

  const [newCategoryName, setNewCategoryName] = useState<string>('');
  const [newProduct, setNewProduct] = useState({
    title: '',
    price: '',
    description: '',
    image_url: '',
    category: '',
  });

  useEffect(() => {
    return () => {
      setIsAdminAuthenticated(false);
    };
  }, []);

  const fetchCategories = async () => {
    const { data } = await supabase
      .from('categories')
      .select('*')
```

File: src/pages/Admin.tsx (continued)

```
      .order('name', { ascending: true });
    if (data) setCategories(data);
  };

  useEffect(() => {
    if (!isAdminAuthenticated) return;

    const fetchAdminData = async () => {
      setLoading(true);

      const { data: ordersData } = await supabase
        .from('orders')
        .select(`
          id,
          user_id,
          total_amount,
          status,
          shipping_address,
          created_at,
          contact_number,
          payment_details,
          shipping_destination,
          order_items(quantity, price_at_purchase, products(id, title))
        `)
        .order('created_at', { ascending: false });

      const { data: productsData } = await supabase
        .from('products')
        .select('*')
        .order('id', { ascending: false });

      await fetchCategories();

      if (ordersData) {
        const typedOrders = ordersData as any[];

        try {
          const userIds = Array.from(new Set(typedOrders.map((o) => o.user_id).filter(Boolean)));
          if (userIds.length > 0) {
            const { data: profilesData } = await supabase
              .from('profiles')
              .select('id, name, email')
              .in('id', userIds);

            if (profilesData) {
              const profileMap = new Map(profilesData.map((p) => [p.id, p]));
              typedOrders.forEach((order) => {
                const match = profileMap.get(order.user_id);
                order.customer_name = match?.name || 'Customer';
                order.customer_email = match?.email || 'N/A';
              });
            }
          }
        } catch (e) {
          console.error('Profile resolution drop context:', getErrorMessage(e));
        }

        setOrders(typedOrders as AdminOrder[]);
      }

      if (productsData) setProducts(productsData);
      setLoading(false);
    };

    fetchAdminData();
  }, [isAdminAuthenticated]);

  const handleAdminLogin = (e: React.FormEvent) => {
    e.preventDefault();
    if (adminPassword === 'supersecretadmin123') {
      setIsAdminAuthenticated(true);
      setAuthError('');
    } else {
      setAuthError(''L Invalid Administrative Password. Access Denied.');
    }
  };

  const handleAdminLogout = () => {
```

File: src/pages/Admin.tsx (continued)

```
155 |     setIsAdminAuthenticated(false);
156 |     setAdminPassword('');
157 |   };
158 |
159 |   const handleUpdateStatus = async (orderId: number, currentStatus: string) => {
160 |     const nextStatus = currentStatus === 'Pending' ? 'Delivered' : 'Pending';
161 |     const { error } = await supabase.from('orders').update({ status: nextStatus }).eq('id', orderId);
162 |     if (!error) {
163 |       setOrders((prev) => prev.map((o) => (o.id === orderId ? { ...o, status: nextStatus } : o)));
164 |     } else {
165 |       alert(`Error updating order status: ${error.message}`);
166 |     }
167 |   };
168 |
169 |   const handleDeleteOrder = async (orderId: number) => {
170 |     if (!confirm(`& þ  Are you sure you want to delete Order #${orderId}?`)) return;
171 |
172 |     try {
173 |       await supabase.from('order_items').delete().eq('order_id', orderId);
174 |       const { error } = await supabase.from('orders').delete().eq('id', orderId);
175 |       if (error) throw error;
176 |
177 |       setOrders((prev) => prev.filter((o) => o.id !== orderId));
178 |     } catch (err: unknown) {
179 |       alert(`Could not erase order: ${getErrorMessage(err)}`);
180 |     }
181 |   };
182 |
183 |   const handleAddCategory = async (e: React.FormEvent) => {
184 |     e.preventDefault();
185 |     if (!newCategoryName.trim()) return;
186 |     setFormLoading(true);
187 |     const { error } = await supabase.from('categories').insert([{ name: newCategoryName.trim() }]);
188 |     if (error) alert(error.message);
189 |     else {
190 |       setNewCategoryName('');
191 |       await fetchCategories();
192 |     }
193 |     setFormLoading(false);
194 |   };
195 |
196 |   const handleDeleteCategory = async (id: string, name: string) => {
197 |     if (!confirm(`Delete category "${name}"?`)) return;
198 |     setFormLoading(true);
199 |     const { error } = await supabase.from('categories').delete().eq('id', id);
200 |     if (error) alert(error.message);
201 |     else await fetchCategories();
202 |     setFormLoading(false);
203 |   };
204 |
205 |   const handleAddProduct = async (e: React.FormEvent) => {
206 |     e.preventDefault();
207 |     if (!newProduct.title || !newProduct.price || !newProduct.image_url || !newProduct.category) {
208 |       alert('Please fill in all saree attributes.');
209 |       return;
210 |     }
211 |     setFormLoading(true);
212 |     const { data, error } = await supabase.from('products').insert([
213 |       {
214 |         title: newProduct.title,
215 |         price: parseFloat(newProduct.price),
216 |         description: newProduct.description,
217 |         image_url: newProduct.image_url,
218 |         category: newProduct.category,
219 |       },
220 |     ]).select();
221 |
222 |     if (error) alert(error.message);
223 |     else if (data) {
224 |       setProducts((prev) => [data[0], ...prev]);
225 |       setNewProduct({ title: '', price: '', description: '', image_url: '', category: '' });
226 |       alert('Ø<ß‰ Saree product published successfully!');
227 |     }
228 |     setFormLoading(false);
229 |   };
230 |
231 |   const handleDeleteProduct = async (productId: number) => {
232 |     if (!confirm('Delete saree product from catalog?')) return;
```

File: src/pages/Admin.tsx (continued)

```
    const { error } = await supabase.from('products').delete().eq('id', productId);
    if (!error) setProducts((prev) => prev.filter((p) => p.id !== productId));
  };

  const triggerSystemPrint = () => {
    window.print();
  };

  if (!isAdminAuthenticated) {
    return (
      <div className="min-h-screen flex flex-col bg-rose-950 font-sans text-white">
        <Navbar />
        <div className="flex-1 flex items-center justify-center p-4">
          <form
            onSubmit={handleAdminLogin}
            className="max-w-md w-full bg-rose-900 border border-rose-800 p-8 rounded-3xl shadow-xl text-
center space-y-4">
            <span className="text-4xl block mb-2">Ø=Ý </span>
            <h1 className="text-xl font-serif font-bold text-amber-200">Merchant Admin Terminal</h1>
            <p className="text-xs text-rose-200/70">Menakkhi Sarees Backoffice Access</p>
            <input
              type="password"
              required
              value={adminPassword}
              onChange={(e) => setAdminPassword(e.target.value)}
              placeholder="Enter Administrative Key..."
              className="w-full text-center text-sm border border-rose-700 bg-rose-950 p-3 rounded-xl
focus:outline-none focus:border-amber-400 text-white font-mono tracking-widest"
            />
            {authError && <p className="text-xs font-bold text-amber-300">{authError}</p>}
            <button
              type="submit"
              className="w-full text-xs font-black uppercase tracking-wider bg-amber-400 text-rose-950 p-3.5
rounded-xl hover:bg-amber-300 transition cursor-pointer"
            >
              Unlock Console
            </button>
          </form>
        </div>
        <Footer />
      </div>
    );
  }

  return (
    <div className="min-h-screen flex flex-col bg-rose-50/20 font-sans relative">
      <style>{`
        @media print {
          body * {
            visibility: hidden !important;
          }
          html, body {
            height: 99vh !important;
            overflow: hidden !important;
            background: #fff !important;
            font-size: 14px !important;
          }
          #drive-invoice-viewport, #drive-invoice-viewport * {
            visibility: visible !important;
          }
          #drive-invoice-viewport {
            position: absolute !important;
            left: 0 !important;
            top: 0 !important;
            width: 100% !important;
            max-width: 100% !important;
            box-shadow: none !important;
            border: none !important;
            padding: 0 !important;
            margin: 0 !important;
          }
          .no-print-action {
            display: none !important;
          }
        }
      `}</style>

      <div className="flex flex-col min-h-screen w-full print:hidden">
        <Navbar />
        <main className="flex-1 max-w-7xl w-full mx-auto px-4 py-8 sm:py-12">
```

File: src/pages/Admin.tsx (continued)

```
          <div className="flex flex-col sm:flex-row justify-between items-start sm:items-center mb-8 gap-4 border-b border-rose-100 pb-6">
            <div>
              <h1 className="text-2xl font-serif font-bold text-rose-950">Menakkhi Sarees Admin Console</h1>
              <button
                onClick={handleAdminLogout}
                className="text-[10px] font-black uppercase tracking-wider bg-rose-100 text-rose-900 px-2.5 py-1 rounded-md mt-1 cursor-pointer"
              >
                Lock Terminal Ø=Ý
              </button>
            </div>
            <div className="flex bg-rose-100/60 p-1 rounded-xl">
              <button
                onClick={() => setActiveTab('orders')}
                className={`px-4 py-2 text-xs font-black uppercase tracking-wider rounded-lg transition cursor-pointer ${
                  activeTab === 'orders' ? 'bg-white text-rose-950 shadow-xs' : 'text-gray-600'
                }`}
              >
                Ø=ÜË Orders ({orders.length})
              </button>
              <button
                onClick={() => setActiveTab('inventory')}
                className={`px-4 py-2 text-xs font-black uppercase tracking-wider rounded-lg transition cursor-pointer ${
                  activeTab === 'inventory' ? 'bg-white text-rose-950 shadow-xs' : 'text-gray-600'
                }`}
              >
                Ø=Üæ Catalog ({products.length})
              </button>
            </div>
          </div>

          {loading ? (
            <div className="text-center py-12 text-xs text-gray-400 animate-pulse font-bold">Loading admin console...</div> ) : activeTab === 'orders' ? (
            <div className="bg-white border border-rose-100 rounded-2xl shadow-xs overflow-x-auto">
              <table className="w-full text-left min-w-[600px]">
                <thead>
                  <tr className="bg-rose-50/50 border-b border-rose-100 text-xs font-black text-rose-950 uppercase tracking-wider"> <th className="p-4 w-[25%]">Ref & Date</th>
                    <th className="p-4 w-[25%]">Customer</th>
                    <th className="p-4 w-[20%] text-center">Amount</th>
                    <th className="p-4 w-[30%] text-center">Actions</th>
                  </tr>
                </thead>
                <tbody className="divide-y divide-rose-50 text-xs sm:text-sm bg-white">
                  {orders.map((order) => (
                    <tr key={order.id} className="hover:bg-rose-50/20 transition">
                      <td className="p-4 whitespace-nowrap">
                        <div className="font-black text-rose-950 font-mono">#{order.id}</div>
                        <div className="text-[11px] text-gray-400 font-bold mt-0.5">
                          {new Date(order.created_at).toLocaleDateString('en-BD')}
                        </div>
                      </td>

                      <td className="p-4">
                        <div className="font-bold text-gray-900">{order.customer_name}</div>
                        <div className="text-xs text-gray-500 font-mono">{order.contact_number}</div>
                      </td>

                      <td className="p-4 text-center font-mono font-black text-rose-950">
                        {formatBDT(order.total_amount)}
                      </td>

                      <td className="p-4 whitespace-nowrap text-center">
                        <div className="flex items-center justify-center gap-2">
                          <button
                            onClick={() => {
                              setActiveInvoiceOrder(order);
                              document.body.style.overflow = 'hidden';
                            }}
                            className="px-3 py-1.5 text-xs font-bold rounded-xl bg-rose-50 text-rose-900 border border-rose-100 hover:bg-rose-100 transition cursor-pointer"
                          >
                            Voucher
                          </button>

                          <button
                            onClick={() => handleUpdateStatus(order.id, order.status)}
                            className={`min-w-[90px] px-2.5 py-1.5 text-xs font-black rounded-xl border transition cursor-pointer ${
                              order.status === 'Delivered'
```

File: src/pages/Admin.tsx (continued)

```
                                ? 'bg-emerald-50 text-emerald-800 border-emerald-200'
                                : 'bg-amber-50 text-amber-800 border-amber-200'
                            }`}
                          >
                            {order.status === 'Delivered' ? 'Ø=ßâ Delivered' : 'Ø=ßá Pending'}
                          </button>

                          <button
                            onClick={() => handleDeleteOrder(order.id)}
                            className="p-1.5 bg-red-50 hover:bg-red-100 rounded-xl text-red-600 text-xs
transition cursor-pointer"
                            title="Delete"
                          >
                            Ø=ÝÑþ
                          </button>
                        </div>
                      </td>
                    </tr>
                  ))}
                </tbody>
              </table>
            </div>
          ) : (
            <div className="grid grid-cols-1 lg:grid-cols-3 gap-8">
              <div className="space-y-6">
                {/* Publish Saree Product */}
                <div className="bg-white border border-rose-100 p-6 rounded-3xl shadow-xs">
                  <h3 className="text-xs font-black uppercase tracking-wider mb-4 text-rose-950">Publish New
Saree</h3>
                  <form onSubmit={handleAddProduct} className="space-y-3">
                    <input
                      type="text"
                      placeholder="Saree Title (e.g. Pure Rajshahi Silk)"
                      required
                      className="w-full text-xs p-2.5 border border-gray-200 rounded-xl bg-gray-50/50"
                      value={newProduct.title}
                      onChange={(e) => setNewProduct({ ...newProduct, title: e.target.value })}
                    />
                    <div className="grid grid-cols-2 gap-3">
                      <input
                        type="number"
                        placeholder="Price in BDT (Ÿ2’
                        step="1"
                        required
                        className="w-full text-xs p-2.5 border border-gray-200 rounded-xl bg-gray-50/50 font-
mono"
                        value={newProduct.price}
                        onChange={(e) => setNewProduct({ ...newProduct, price: e.target.value })}
                      />
                      <select
                        required
                        className="w-full text-xs p-2.5 border border-gray-200 rounded-xl bg-gray-50/50 font-
bold text-gray-700"
                        value={newProduct.category}
                        onChange={(e) => setNewProduct({ ...newProduct, category: e.target.value })}
                      >
                        <option value="">-- Category --</option>
                        {categories.map((c) => (
                          <option key={c.id} value={c.name}>
                            {c.name}
                          </option>
                        ))}
                      </select>
                    </div>
                    <input
                      type="url"
                      placeholder="Image URL"
                      required
                      className="w-full text-xs p-2.5 border border-gray-200 rounded-xl bg-gray-50/50"
                      value={newProduct.image_url}
                      onChange={(e) => setNewProduct({ ...newProduct, image_url: e.target.value })}
                    />
                    <textarea
                      placeholder="Saree Weave Description & Details"
                      rows={3}
                      className="w-full text-xs p-2.5 border border-gray-200 rounded-xl bg-gray-50/50 resize-
none"
                      value={newProduct.description}
                      onChange={(e) => setNewProduct({ ...newProduct, description: e.target.value })}
                    />
                    <button
                      type="submit"
                      disabled={formLoading}
```

File: src/pages/Admin.tsx (continued)

```
467 |                       className="w-full text-xs font-black uppercase tracking-wider bg-rose-900 text-white
p-3 rounded-xl hover:bg-rose-800 transition disabled:opacity-50 cursor-pointer"
468 |                     >
469 |                       Ø=Þ€ Publish Live Saree
470 |                     </button>
471 |                   </form>
472 |                 </div>
473 |
474 |                 {/* Manage Categories */}
475 |                 <div className="bg-white border border-rose-100 p-6 rounded-3xl shadow-xs">
476 |                   <h3 className="text-xs font-black uppercase tracking-wider mb-3 text-rose-950">Add Saree
Category</h3>
477 |                   <form onSubmit={handleAddCategory} className="flex gap-2 mb-4">
478 |                     <input
479 |                       type="text"
480 |                       placeholder="Category name..."
481 |                       required
482 |                       className="flex-1 text-xs p-2 border border-gray-200 rounded-xl bg-gray-50/50"
483 |                       value={newCategoryName}
484 |                       onChange={(e) => setNewCategoryName(e.target.value)}
485 |                     />
486 |                     <button
487 |                       type="submit"
488 |                       disabled={formLoading}
489 |                       className="text-xs font-black uppercase bg-rose-950 text-white px-4 rounded-xl
hover:bg-rose-800 transition disabled:opacity-50 cursor-pointer"
490 |                     >
491 |                       Add
492 |                     </button>
493 |                   </form>
494 |                   <div className="space-y-1.5 max-h-40 overflow-y-auto">
495 |                     {categories.map((c) => (
496 |                       <div
497 |                         key={c.id}
498 |                         className="flex justify-between items-center bg-rose-50/50 border border-rose-100
p-2 rounded-xl text-xs font-bold text-rose-950"
499 |                       >
500 |                         <span>{c.name}</span>
501 |                         <button
502 |                           type="button"
503 |                           onClick={() => handleDeleteCategory(c.id, c.name)}
504 |                           className="text-red-500 hover:underline cursor-pointer"
505 |                         >
506 |                           Remove
507 |                         </button>
508 |                       </div>
509 |                     ))}
510 |                   </div>
511 |                 </div>
512 |               </div>
513 |
514 |               {/* Saree Catalog Records */}
515 |               <div className="lg:col-span-2 space-y-3">
516 |                 <h3 className="text-xs font-black uppercase tracking-wider text-rose-950 mb-2">
517 |                   Active Saree Catalog ({products.length})
518 |                 </h3>
519 |                 {products.map((product) => (
520 |                   <div key={product.id} className="bg-white border border-rose-100 p-4 rounded-2xl shadow-xs
flex items-center justify-between">
521 |                     <div className="flex items-center gap-4">
522 |                       <img src={product.image_url} alt={product.title} className="w-12 h-12 rounded-xl
object-contain bg-rose-50/50 p-1" />
523 |                       <div>
524 |                         <h4 className="font-bold text-rose-950 text-sm leading-none">{product.title}</h4>
525 |                         <div className="flex items-center gap-2 mt-1.5">
526 |                           <span className="text-xs font-black text-rose-800 font-
mono">{formatBDT(product.price)}</span>
527 |                           <span className="text-[9px] uppercase font-black bg-rose-100 text-rose-900 px-2
py-0.5 rounded">
528 |                             {product.category}
529 |                           </span>
530 |                         </div>
531 |                       </div>
532 |                     </div>
533 |                     <button
534 |                       onClick={() => handleDeleteProduct(product.id)}
535 |                       className="text-gray-400 hover:text-red-600 p-2 text-xs cursor-pointer"
536 |                     >
537 |                       Ø=ÝÑþ
538 |                     </button>
539 |                   </div>
540 |                 ))}
541 |               </div>
542 |             </div>
543 |           )}
544 |         </main>
```

File: src/pages/Admin.tsx (continued)

```
        <Footer />
      </div>

      {/* ADMIN PRINTABLE INVOICE MODAL */}
      {activeInvoiceOrder && (
        <div className="fixed inset-0 z-50 bg-white flex flex-col justify-start items-center overflow-y-auto p-2 sm:p-6 select-none animate-in fade-in duration-200">
          <div id="no-print-action-bar" max-w-4xl w-[95vw] bg-rose-950 text-white rounded-t-2xl px-4 py-3 border-b border-rose-800 flex justify-between items-center gap-2 shadow-xl">
            <div className="flex items-center gap-2.5 truncate">
              <span className="text-xl shrink-0">Ø=ÜÄ</span>
              <div className="truncate text-left">
                <p className="text-xs sm:text-sm font-bold truncate">Menakkhi_Order_#000{activeInvoiceOrder.id}.pdf</p>
                <p className="text-[10px] text-rose-200/70 font-semibold">Official Admin Invoice</p>
              </div>
            </div>
            <div className="flex items-center gap-2 shrink-0">
              <button
                onClick={triggerSystemPrint}
                className="px-4 py-1.5 bg-amber-400 hover:bg-amber-300 text-rose-950 font-black text-xs uppercase tracking-wider rounded-xl transition cursor-pointer shadow-md"
              >
                Ø=Üå Print Invoice
              </button>
              <button
                onClick={() => {
                  setActiveInvoiceOrder(null);
                  document.body.style.overflow = 'unset';
                }}
                className="p-1.5 bg-rose-900 hover:bg-rose-800 rounded-xl transition cursor-pointer text-sm text-rose-200"
                title="Close"
              >
                '
              </button>
            </div>
          </div>

          <div
            id="drive-invoice-viewport"
            className="max-w-4xl w-[95vw] bg-white text-black p-4 sm:p-12 shadow-sm rounded-b-2xl min-h-[70vh] flex flex-col justify-between font-sans leading-relaxed text-sm tracking-normal border border-rose-100 border-t-0"
          >
            <div>
              <div className="flex flex-col sm:flex-row justify-between items-start border-b-4 border-rose-950 pb-4 mb-6 gap-4">
                <div>
                  <h1 className="text-2xl sm:text-4xl font-serif font-bold uppercase tracking-tight text-rose-950">MENAKKHI SAREES</h1>
                  <p className="text-xs font-bold text-rose-800 mt-0.5">Admin Dispatch Statement</p>
                </div>
                <div className="text-left sm:text-right">
                  <h2 className="text-xl sm:text-2xl font-black text-gray-900 tracking-tight">INVOICE RECORD</h2>
                  <p className="text-sm sm:text-base font-black text-rose-900 mt-0.5">Order Sequence: #000{activeInvoiceOrder.id}</p>
                  <p className="text-xs text-gray-400 font-bold mt-1">
                    Date: {new Date(activeInvoiceOrder.created_at).toLocaleDateString('en-BD')}
                  </p>
                </div>
              </div>

              <div className="grid grid-cols-1 sm:grid-cols-2 gap-4 mb-6 bg-rose-50/40 border border-rose-100 rounded-2xl p-4">
                <div>
                  <h3 className="text-[10px] font-black text-rose-900 uppercase tracking-widest mb-1">Customer Identity</h3>
                  <p className="text-sm font-black text-gray-900">{activeInvoiceOrder.customer_name || 'Customer'}</p>
                  <p className="text-xs text-gray-600 mt-0.5">' þ  {activeInvoiceOrder.customer_email || 'N/A'}</p>
                  <p className="text-xs font-mono font-bold text-rose-900 mt-1">Ø=ÜÞ {activeInvoiceOrder.contact_number || 'N/A'}</p>
                </div>
                <div>
                  <h3 className="text-[10px] font-black text-rose-900 uppercase tracking-widest mb-1">Delivery Destination</h3>
                  <p className="text-xs text-gray-800 font-medium leading-relaxed">
                    {activeInvoiceOrder.shipping_address}
                  </p>
                </div>
              </div>

              <div className="mb-6">
                <h3 className="text-[10px] font-black text-rose-900 uppercase tracking-widest mb-2.5">Line Items</h3>
                <table className="w-full text-left text-xs sm:text-sm">
                  <thead>
                    <tr className="border-b-2 border-rose-950 font-black text-rose-950 bg-rose-50">
                      <th className="py-2.5 px-3 w-[50%]">Saree Title</th>
                      <th className="py-2.5 px-3 text-center w-[15%]">Qty</th>
                      <th className="py-2.5 px-3 text-right w-[35%]">Net Total</th>
                    </tr>
                  </thead>
```

File: src/pages/Admin.tsx (continued)

```
623 |                   <tbody className="divide-y divide-rose-50">
624 |                     {activeInvoiceOrder.order_items.map((item, index) => (
625 |                       <tr key={index} className="font-semibold text-gray-800">
626 |                         <td className="py-3 px-3 font-bold text-rose-950">{item.products?.title || 'Saree
Item'}</td>
627 |                         <td className="py-3 px-3 text-center font-mono">{item.quantity}</td>
628 |                         <td className="py-3 px-3 text-right font-mono font-black text-rose-950">
629 |                           {formatBDT(item.quantity * item.price_at_purchase)}
630 |                         </td>
631 |                       </tr>
632 |                     ))}
633 |                   </tbody>
634 |                 </table>
635 |               </div>
636 |             </div>
637 |
638 |             <div className="mt-auto pt-6 border-t-2 border-rose-950">
639 |               <div className="flex flex-col sm:flex-row justify-between items-start sm:items-end gap-4">
640 |                 <div>
641 |                   <h3 className="text-[10px] font-black text-rose-900 uppercase tracking-widest
mb-1">Payment Clearance</h3>
642 |                   <span className="text-xs font-black bg-rose-100 text-rose-950 border border-rose-200 px-3
py-1 rounded-xl uppercase">{activeInvoiceOrder.payment_details?.method === 'COD'
643 |                     ? 'Ø=Üµ Cash On Delivery'
644 |                     : activeInvoiceOrder.payment_details?.method === 'bKash'
645 |                     ? 'Ø<ß8 bKash Personal'
646 |                     : 'Ø=Ý% Nagad Personal'}
647 |                   </span>
648 |                   {activeInvoiceOrder.payment_details?.trx_id && activeInvoiceOrder.payment_details.method !
==='COD' && (
649 |                     <p className="text-xs font-mono font-bold text-gray-600 mt-2">
650 |                       TrxID: {activeInvoiceOrder.payment_details.trx_id}
651 |                     </p>
652 |                   )}
653 |                 </div>
654 |
655 |                 <div className="text-right w-full sm:w-auto min-w-[200px] space-y-1">
656 |                   <div className="flex justify-between items-end pt-2 border-t border-gray-200">
657 |                     <span className="text-xs font-black text-rose-950 uppercase tracking-wider">Grand
Total</span>
658 |                     <span className="text-xl font-black text-rose-950 font-mono">
659 |                       {formatBDT(activeInvoiceOrder.total_amount)}
660 |                     </span>
661 |                   </div>
662 |                 </div>
663 |               </div>
664 |
665 |               <div className="mt-8 text-center text-[9px] text-gray-400 font-bold border-t border-rose-50
pt-3 uppercase tracking-widest">Mehakkhi Sarees Merchant Copy Record
666 |               </div>
667 |             </div>
668 |           </div>
669 |         </div>
670 |       )}
671 |     </div>
672 |   );
673 | }
```

File: src/index.css

```
1 | @import "tailwindcss";
```

File: src/App.css

```
.counter {
  font-size: 16px;
  padding: 5px 10px;
  border-radius: 5px;
  color: var(--accent);
  background: var(--accent-bg);
  border: 2px solid transparent;
  transition: border-color 0.3s;
  margin-bottom: 24px;

  &:hover {
    border-color: var(--accent-border);
  }
  &:focus-visible {
    outline: 2px solid var(--accent);
    outline-offset: 2px;
  }
}

.hero {
  position: relative;

  .base,
  .framework,
  .vite {
    inset-inline: 0;
    margin: 0 auto;
  }

  .base {
    width: 170px;
    position: relative;
    z-index: 0;
  }

  .framework,
  .vite {
    position: absolute;
  }

  .framework {
    z-index: 1;
    top: 34px;
    height: 28px;
    transform: perspective(2000px) rotateZ(300deg) rotateX(44deg) rotateY(39deg)
      scale(1.4);
  }

  .vite {
    z-index: 0;
    top: 107px;
    height: 26px;
    width: auto;
    transform: perspective(2000px) rotateZ(300deg) rotateX(40deg) rotateY(39deg)
      scale(0.8);
  }
}

#center {
  display: flex;
  flex-direction: column;
  gap: 25px;
  place-content: center;
  place-items: center;
  flex-grow: 1;

  @media (max-width: 1024px) {
    padding: 32px 20px 24px;
    gap: 18px;
  }
}

#next-steps {
  display: flex;
  border-top: 1px solid var(--border);
  text-align: left;
```

File: src/App.css (continued)

```
 77 |
 78 |   & > div {
 79 |     flex: 1 1 0;
 80 |     padding: 32px;
 81 |     @media (max-width: 1024px) {
 82 |       padding: 24px 20px;
 83 |     }
 84 |   }
 85 |
 86 |   .icon {
 87 |     margin-bottom: 16px;
 88 |     width: 22px;
 89 |     height: 22px;
 90 |   }
 91 |
 92 |   @media (max-width: 1024px) {
 93 |     flex-direction: column;
 94 |     text-align: center;
 95 |   }
 96 | }
 97 |
 98 | #docs {
 99 |   border-right: 1px solid var(--border);
100 |
101 |   @media (max-width: 1024px) {
102 |     border-right: none;
103 |     border-bottom: 1px solid var(--border);
104 |   }
105 | }
106 |
107 | #next-steps ul {
108 |   list-style: none;
109 |   padding: 0;
110 |   display: flex;
111 |   gap: 8px;
112 |   margin: 32px 0 0;
113 |
114 |   .logo {
115 |     height: 18px;
116 |   }
117 |
118 |   a {
119 |     color: var(--text-h);
120 |     font-size: 16px;
121 |     border-radius: 6px;
122 |     background: var(--social-bg);
123 |     display: flex;
124 |     padding: 6px 12px;
125 |     align-items: center;
126 |     gap: 8px;
127 |     text-decoration: none;
128 |     transition: box-shadow 0.3s;
129 |
130 |     &:hover {
131 |       box-shadow: var(--shadow);
132 |     }
133 |     .button-icon {
134 |       height: 18px;
135 |       width: 18px;
136 |     }
137 |   }
138 |
139 |   @media (max-width: 1024px) {
140 |     margin-top: 20px;
141 |     flex-wrap: wrap;
142 |     justify-content: center;
143 |
144 |     li {
145 |       flex: 1 1 calc(50% - 8px);
146 |     }
147 |
148 |     a {
149 |       width: 100%;
150 |       justify-content: center;
151 |       box-sizing: border-box;
152 |     }
153 |   }
154 | }
```

File: src/App.css (continued)

```
155 |
156 | #spacer {
157 |   height: 88px;
158 |   border-top: 1px solid var(--border);
159 |   @media (max-width: 1024px) {
160 |     height: 48px;
161 |   }
162 | }
163 |
164 | .ticks {
165 |   position: relative;
166 |   width: 100%;
167 |
168 |   &::before,
169 |   &::after {
170 |     content: '';
171 |     position: absolute;
172 |     top: -4.5px;
173 |     border: 5px solid transparent;
174 |   }
175 |
176 |   &::before {
177 |     left: 0;
178 |     border-left-color: var(--border);
179 |   }
180 |   &::after {
181 |     right: 0;
182 |     border-right-color: var(--border);
183 |   }
184 | }
```